RÔMULO MILANI DOMINGOS

MINERAÇÃO DE TEXTO EM LÍNGUA PORTUGUESA

Trabalho de Conclusão de Curso apresentado como exigência parcial, para a obtenção do grau no curso de Ciência da Computação, da Universidade de Franca.

Orientadora: Profa. Claudia Vicci Amadeu

FRANCA
2014

DEDICO aos meus pais, Maria Aparecida Milani Domingos e Donizeti Francisco Domingos, meus exemplos de caráter, força e dignidade. Sem dúvida, os maiores mestres que já tive.

AGRADEÇO a meus irmãos, Henrique e Débora por todo amor; à minha orientadora Claudia Vicci Amadeu, pelo esforço e apoio nessa jornada; à minha namorada Mirella Souza Oliveira, pela compreensão e carinho; a todos os meus amigos que sempre me ajudaram, torceram e acreditaram em mim.

“O maior talento que alguém pode ter é a dedicação.”

José Tiago Sabino

RESUMO

DOMINGOS, Rômulo Milani. **Mineração de texto em língua portuguesa.** 2014. 78 f. Trabalho de Conclusão de Curso (Graduação em Ciência da Computação) – Universidade de Franca, Franca.

Estipula-se que hoje em dia, cerca de 80% da internet se encontra em modo texto. Documentos, portais de notícias e até mesmo as próprias páginas da web (HTML), encontram-se em formato textual. O objetivo deste trabalho é o estudo do processo de mineração de texto e de todas as tecnologias que fazem parte de seu contexto, passando de forma detalhada por todas as etapas que compõem a tecnologia. Para a ilustração do trabalho, foi proposta a implementação de um protótipo de classificador de opinião, apresentando uma das partes fundamentais de um sistema de mineração e dados textuais, a classificação. Para a implementação do protótipo foi utilizada a linguagem Python 2.7.6, uma linguagem dinâmica e de alto nível. Os dados, tanto de treinamento quanto de testes, foram obtidos da rede social Twitter, por meio da biblioteca TwitterSearch 0.78.4. A classificação dos textos foi realizada com o algoritmo de *Naive Bayes*, algoritmo de aprendizagem de máquina que baseia sua classificação em estatística. Para a implementação do algoritmo de classificação *Naive Bayes* foi utilizado o *kit* de desenvolvimento de aplicação de linguagem natural NLTK 3.0. Por fim, a apresentação do resultado foi feita por meio dos gráficos da biblioteca Matplotlib 1.4.2. Assim, o protótipo final será capaz de ler, processar e classificar os textos de forma automática.

Palavras-chave: *Mineração de texto*; *Python*; *NLTK; Naive Bayes.*

ABSTRACT

DOMINGOS, Rômulo Milani. **Mineração de texto em língua portuguesa.** 2014. 78 f. Trabalho de Conclusão de Curso (Graduação em Ciência da Computação) – Universidade de Franca, Franca.

It is stipulated that today, about 80% of the internet is in text mode, documents, news portals and even their own web pages (HTML), are in textual format. The objective of this work is the study of text mining and all the technologies that are part of its context, passing in detail all the stages that compose the technology. For illustration of the work, it has been proposed to implement a prototype classifier view, showing essential parts of a system of a mining and textual data, the classification. For the prototype implementation of the language Python 2.7.6 was used, a dynamic and high-level language. The data for both training as testing, were obtained from the social network Twitter by Twitter Search 0.78.4 library. The classification of the texts was performed using the Naive Bayes algorithm, machine learning algorithm that is based on statistical classification. For the implementation of the Naive Bayes classification the development kit for application of natural language NLTK 3.0 was used. Finally, the presentation of the results was performed using the graphics library Matplotlib 1.4.2. Therefore the final prototype will be able to read, process and classify texts automatically.

Key words: *Text Mining*; *Python*; *NLTK; Naive Bayes.*

LISTA DE ABREVIATURAS

KDD	*Knowledge Dyscovery in Databeses*
NLTK	*Natural Language Toolkit*
PLN	Processamento de Linguagem Natural
RI	Recuperação da Informação
DW	*Data Warehouse*
HTML	*HyperText Markup Language*
URL	*Uniform Resource Locator*
SVM	*Support Vector Machines*
MT	*Mineração de Textos*
MD	*Mineração de Dados*
API	*Application Programming Interface*
RT	*Retweets*

LISTA DE FIGURAS

Figura 1	-	Processos de KDD.	21
Figura 2	-	Processos da mineração de texto, proposto por Aranha (2007).	22
Figura 3	-	Sites brasileiros quanto à frequência de alteração do conteúdo.	24
Figura 4	-	Os três gráficos (1), (2) e (3) mostram, de forma ilustrativa, a necessidade de aplicação de três tarefas diferentes de PLN T1, T2 e T3. T2 só consegue 90% de acerto de melhorar T1 e T3.	28
Figura 5	-	Componentes de um sistema de Recuperação de Informação.	29
Figura 6	-	Modelo simples de aprendizagem de máquina.	30
Figura 7	-	Teorema de *Naive Bayes*.	31
Figura 8	-	Funcionamento de um *crawler* simples.	35
Figura 9	-	Comparação entre *crawler* focado e *crawler* comum.	36
Figura 10	-	Relacionamentos de co-citação e tunelamento em uma comunidade *web*.	38
Figura 11	-	Representação do modelo *Bag-of-Words*.	45
Figura 12	-	Representação de uma lista invertida.	46
Figura 13	-	Funcionamento básico de um dicionário *Thesaurus*.	47
Figura 14	-	Processo de categorização.	49
Figura 15	-	Processo básico de clusterização.	50
Figura 16	-	Processo de extração de informação.	52
Figura 17	-	Programa “Olá Mundo” em Python.	55
Figura 18	-	Método *replace* em Python.	56
Figura 19	-	Método *split* em Python.	56
Figura 20	-	Método *join* em Python.	56
Figura 21	-	Métodos *lower*, *upper* e *capitalize* em Python.	57

Figura 22 - Elementos tipo lista em Python. 57

Figura 23 - Manipulando lista em Python. 58

Figura 24 - Tuplas em Python. 58

Figura 25 - Dicionários em Python. 59

Figura 26 - Manipulando elemento do dicionário. 59

Figura 27 - Métodos *keys*, *values* e *items* do tipo dicionário. 59

Figura 28 - Tipos de *import* da biblioteca NLTK. 60

Figura 29 - Método *word_tokenize* da biblioteca NLTK. 60

Figura 30 - Método *word_tokenize* da biblioteca NLTK. 61

Figura 31 - Exemplo de um *tweet*. 62

Figura 32 - Método pegaTweets da classe Spyder. 63

Figura 33 - Método processatweet da classe processamentoTexto. 64

Figura 34 - Método removestopwords da classe processamentoTexto. 64

Figura 35 - Método tokenizacao da classe processamentoTexto. 64

Figura 36 - Método remover_acentos da classe processamentoTexto. 65

Figura 37 - Modelo utilizado pela classificação supervisionada. 65

Figura 38 - Lista de tuplas contendo tweets positivo. 66

Figura 39 - Lista de tuplas contendo tweets negativos. 66

Figura 40 - Método que retira todas as palavras da lista de treinamento. 67

Figura 41 - Método retorna as 2000 palavras mais frequentes. 67

Figura 42 - Dicionário resultado do método *FreqDist* do NLTK. 67

Figura 43 - Método de extração de características. 68

Figura 44 - Dicionário resultado do método extrair_caracteristicas. 68

Figura 45 - Conjunto de treinamento, retorno do método *apply_features.* 69

Figura 46 - Treinando o classificador *Naive Bayes*. 69

Figura 47 - Características relevantes do classificador. 70

Figura 48 - Método para o cálculo de precisão do classificador. 70

Figura 49 - Classificação de um *tweet*. 71

Figura 50 - Análise de sentimentos referentes a reeleição de Dilma Rousseff. 72

Figura 51 - Pesquisa de intenção de votos. 73

Figura 52 - Pesquisa de intenção de votos. 73

LISTA DE TABELAS

Tabela 1 - Análise manual versus análise do algoritmo. 71

LISTA DE QUADROS

Quadro 1	-	As duas abordagens para a análise de textos e suas principais áreas de conhecimento.	27
Quadro 2	-	As principais características de cada uma das abordagens para a análise de textos.	27
Quadro 3	-	Representação atributo-valor obtida a partir da etapa de pré-processamento.	39
Quadro 4	-	Exemplificação do resultado da execução de um subsistema de *tokenization* que se baseia em dicionários pré-estabelecidos e regras de formação.	41
Quadro 5	-	Identificação e remoção de *stopwords*.	43

SUMÁRIO

INTRODUÇÃO 18
1 MINERAÇÃO DE TEXTO 20
1.1 DESCOBERTA DE CONHECIMENTO EM BASE DE DADOS 20
1.2 INTRODUÇÃO À MINERAÇÃO DE TEXTOS 21
1.3 MINERAÇÃO DE TEXTO: FUNDAMENTOS 23
1.3.1 Principais Elementos 23
1.3.2 Características representativas de um documento 24
1.4 TIPOS DE PROCESSOS DE MINERAÇÃO DE TEXTO 26
1.4.1 Análise Estatística 26
1.4.2 Análise Semântica 26
1.5 ÁREAS CORRELATAS À MINERAÇÃO DE TEXTO 27
1.5.1 Ciência Cognitiva 27
1.5.2 Processamento de linguagem natural 28
1.5.3 Recuperação da informação 29
1.5.4 Aprendizado de máquina 30
1.5.5 Estatística 31
1.5.6 Mineração de dados 32
2 ETAPAS DA MINERAÇÃO DE TEXTO 33
2.1 COLETA DE INFORMAÇÕES 33
2.1. Crawlers 34
2.1.2 Crawlers focados 36
2.2 PRÉ-PROCESSAMENTO 38
2.2.1 Tokenização 39
2.2.2 Redução do léxico 42
2.2.3 Seleção de características 42
2.2.4 Remoção de Stopwords 42
2.2.5 Normalização 43
2.3 INDEXAÇÃO 45

2.3.1 Listas invertidas 46
2.3.2 Indexação temática 47
2.4 MINERAÇÃO DE TEXTO 47
2.4.1 Categorização de texto 48
2.4.2Clusterização 50
2.4.3 Sumarização 51
2.4.4 Extração de informação 52
2.5 ANÁLISE 53
3 CLASSIFICADOR DE TWEETS 54
3.1 ALGORIMTO DE CLASSIFICAÇÃO 54
3.2 LINGUAGEM DE PROGRAMAÇÃO PHYTON 54
3.3 NATURAL LANGUAGE TOOLKIT (NLTK) 60
3.4 PROTÓTIPO DE CLASSIFICADOR DE TWEETS 61
3.4.1 Coletando tweets 62
3.4.2 Processando os tweets 63
3.4.3 Classificação supervisionada 65
3.4.4 Base de treinamento 66
3.4.5 Implementação do classificador 66
3.4.6 Resultados obtidos 68
CONCLUSÃO 74
REFERÊNCIAS 76

INTRODUÇÃO

Com avanço da computação, o fluxo de informações cresceu consideravelmente, gerando bancos de dados com enorme capacidade de armazenamento. Tendo em vista a necessidade de buscar informações ocultas sobre esses dados, teve-se a concepção do processo de descobrimento de conhecimento em base de dados, o KDD (*Knowledge Dyscovery in Databases*).

Paralelamente a isso, a internet se tornou uma enorme fonte de informações, sendo que essas não são armazenadas, em sua maioria, de forma estruturada, como em um banco de dados. Encontram-se em modo texto, o que não permite a execução da sequência tradicional da mineração de dados, pois não são dados estruturados. Para a resolução desse tipo de problema, foi criada a técnica de mineração de textos, que permite executar o processo de mineração sobre esses dados.

Assim, esse trabalho tem, como objetivo, a apresentação do processo de mineração de textos, suas particularidades e as tecnologias que o abrangem. Para exemplificação, será desenvolvido um protótipo de classificador de opinião, que será aplicado na rede social Twitter. Esse classificador terá, como função, avaliar se um *tweet* tem uma mensagem positiva ou negativa sobre um tema específico.

Para o desenvolvimento do classificador será necessária a criação uma base de treinamento, que será feita na linguagem de programação Phyton junto à biblioteca TwitterSearch, para acesso aos *tweets*. Tendo a base de treinamento consolidada, será utilizado o *kit* de desenvolvimento NLTK (*Natural Language Toolkit*), que terá como função aplicar o algoritmo de classificação de *Naive Bayes*, que será o responsável por classificar os *tweets*. Para melhor visualização dos resultados, será utilizada a biblioteca Matplotlib para geração de gráficos.

Espera-se que o trabalho consiga atingir os objetivos, bem como ajudar no entendimento da mineração de texto.

O trabalho será dividido em três capítulos: Mineração de Texto, Etapas da Mineração de Texto e Classificador de Tweets. No capítulo de Mineração de Texto, será abordado de uma forma geral a mineração de texto e suas tecnologias. O capítulo de Etapas da

Mineração de Texto abordará de uma forma detalhada todas as etapas do processo de mineração de texto. E por fim no capítulo Classificador de Tweets, será demonstrado o desenvolvimento do classificador de opinião, bem como todas as tecnologias e técnicas que foram utilizadas nesse processo.

1 MINERAÇÃO DE TEXTO

Esse capítulo tem como objetivo apresentar uma visão global do processo de mineração de texto, bem como as tecnologias que abrangem esse processo

1.1 DESCOBERTA DE CONHECIMENTO EM BASE DE DADOS

"Estamos nos afogando em informações, mas com sede de conhecimento" (SANTOS, 2009). Essa frase mostra bem o cenário atual, onde possuímos enormes bases de dados, mas não extraímos o real valor desses dados. Visando automatizar a busca por informações ocultas, surge um novo conjunto de ferramentas e métodos para auxiliar essa busca, o KDD.

O processo de descoberta de conhecimento em bases de dados, KDD ou DCBD, é a metodologia global para se descobrir informações úteis em bases de dados. Surgiu no final da década de 1980, resultado da expansão das bases de dados computacionais da época (SOARES, 2008).

De acordo com Fayyad (1996), o processo de KDD é composto por um conjunto de etapas que vão da análise do problema até a interpretação do conhecimento obtido.

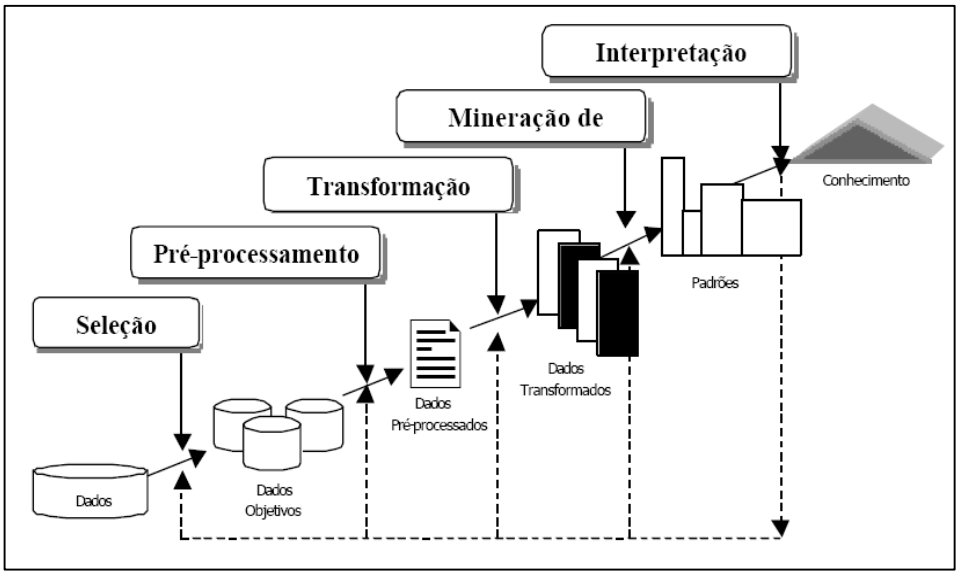

Figura 1 – Processos de KDD

Fonte: FAYYAD, 1996, p.33.

Pela Figura 1, é possível acompanhar todo o processo da descoberta de conhecimento em bases de dados, que se estende do conjunto de dados até a interpretação dos resultados. Pode-se observar, também, sua interatividade, já que podemos voltar a qualquer etapa do processo e realizá-la novamente.

Todos esses processos realizados pelo KDD, que em princípio foram utilizados em bancos de dados estruturados, servem de parâmetros para a aplicação da mineração de texto, que faz uso de técnicas próprias, por tratar dados não estruturados, conforme o que se apresenta, a seguir.

1.2 INTRODUÇÃO À MINERAÇÃO DE TEXTO

Todos os dias milhares de documentos trafegam na internet e grandes massas de dados são geradas a todo o momento, fazendo da internet a maior base de dados do mundo. A proliferação de documentos disponíveis em sites, *intranets* e servidores aumenta de forma esmagadora. Segundo Aranha (2011), cerca de 80% destas informações estão em modo texto, e a necessidade de utilizar esses dados textuais de forma produtiva deu início, então, a uma nova linha de pesquisa: a mineração de texto.

Mineração de Texto, Mineração de Dados Textuais ou *Text Mining*, segundo Lucas (2007), "é uma forma de examinar uma coleção de documentos e descobrir informação não contida em nenhum dos documentos." Sanger e Feldman (2006) descrevem o *Text Mining* como "processo de conhecimento intensivo nos quais um usuário interage com uma coleção de documentos ao longo do tempo usando um conjunto de ferramentas de análise".

A partir das definições dadas pelos autores citados, nota-se que o principal objetivo do *Text Mining* é extrair informações relevantes, diante de um conjunto de documentos textuais, técnica que se assemelha à mineração de dados. Porém, o que as difere é exatamente o tipo de dados que cada uma trabalha. A mineração de dados trabalha com extração de padrões de dados estruturados, ou seja, dados que já receberam tratamento e estão alocados em um banco de dados. Já a mineração de texto aplica suas técnicas em um conjunto de dados não estruturados, em modo texto.

Por se tratar de dados não estruturados, a mineração de texto apresenta um processo de tratamento destes dados antes da aplicação dos algoritmos de extração de conhecimento.

A Figura 2 apresenta o esquema proposto por Aranha (2007), onde o processo de mineração de texto aparece em cinco etapas.

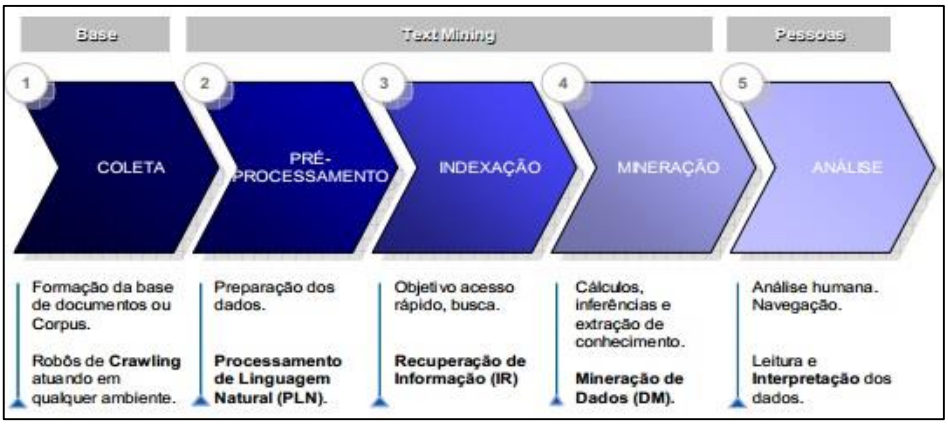

Figura 2 – Processos da Mineração de texto, proposto por Aranha (2007).

Fonte: ARANHA, 2007, p.19.

A coleta é a primeira fase do processo do *Text Mining*. Nesta etapa são selecionados e capturados os textos que irão compor a base de textos, também conhecido como *corpus*.

Logo após a coleta, tem-se a etapa de pré-processamento, na qual são aplicados vários algoritmos que darão uma forma mais estruturada à base de dados textuais.

O processo de indexação é responsável pela organização dos dados. São criados índices que serão usados para a otimização do banco, para que o processo de recuperação de dados seja mais rápido.

Após as etapas anteriores estarem concluídas, são aplicados os algoritmos na etapa de mineração, processo em que ocorre a extração do conhecimento das bases de dados textuais.

Enfim, na etapa de análise, os conhecimentos obtidos são analisados e interpretados pelo profissional responsável pela área em que a mineração foi direcionada.

1.3 MINERAÇÃO DE TEXTO: FUNDAMENTOS

1.3.1 Principais elementos

O principal elemento de um processo de MT é a coleção de documentos, pois eles constituem o conjunto de dados sob o qual este processo é realizado. Um documento, elemento básico de uma coleção, é definido como uma unidade discreta de dados de forma textual, como por exemplo, uma página *web* ou um *e-mail* (KONCHADY, 2006).

Segundo Soares (2008), em alguns cenários, a coleção de documentos pode ser estática, ou seja, o conjunto de documentos permanece inalterado tanto em elementos, como em conteúdo. Entretanto, em grande parte dos casos, essa coleção é dinâmica, com seus elementos sendo incluídos, excluídos e até mesmo alterados, o que demanda desafios ainda maiores para um sistema de MT.

A Figura 3, mostra a divisão dos sites brasileiros de acordo com alteração do conteúdo.

Figura 3: Sites brasileiros quanto à frequência da alteração de conteúdo.
Fonte: SOARES, 2008, p.27.

1.3.2 Características representativas de um documento

As operações de pré-processamento envolvidas em MT possuem como objetivo realçar os diferentes elementos presentes em um documento em linguagem natural, para transformá-lo de uma representação estrutural irregular e implícita, a uma representação explicitamente estruturada (SOARES, 2008).

Entretanto, dado o número potencialmente enorme de palavras, frases, sentenças, elementos tipográficos e *tags* de layout que, até mesmo um simples e pequeno documento pode apresentar uma tarefa essencial para qualquer sistema de MT. Essa tarefa consiste na identificação do conjunto mais simples de características de um documento que pode representá-lo como um todo, denominando-se representação do modelo.

Para Aranha (2007) "o modelo Espaço Vetorial é uma das técnicas mais utilizadas em mineração de texto, sendo a aplicação mais comum à classificação automática de documentos". Segundo Carrilho (2007), documentos são representados como pontos em um espaço Euclidiano t-dimensional em que cada dimensão corresponde a um *token* do léxico. Desta forma, D_i diz respeito ao i-ésimo componente do documento D, o qual possui um peso associado.

Fazendo um comparativo entre a MT e a MD, problemas relacionados à dimensionalidade estão mais presentes na MT. Isso se deve ao fato das inúmeras

possibilidades existentes para a seleção de características de um documento com informações textuais (SOARES, 2008).

No processo de construção de um modelo de representação há dois objetivos essenciais:

- Primeiro: determinar uma quantidade de informações suficientes para que se possam representar os documentos, sem que haja grandes perdas de informações semânticas, o que gera um número grande de características.
- Segundo: com base nessas informações, determinar o menor número possível de características para que os modelos de representação sejam computacionalmente eficientes.

Muitos critérios podem ser utilizados na seleção dos tipos de características que vão representar um documento. Segundo Soares (2008), as três seguintes são as mais utilizadas:

- **Caracteres**: componentes individuais que são responsáveis por um nível semântico maior, com palavras e termos. Embora a construção de um modelo que utilize estas características seja considerado mais próximo da realidade, a alta dimensionalidade torna inviável a utilização de diversas técnicas computacionais.
- **Palavras**: são a menor unidade capaz de representar algum valor semântico de um documento. Sendo assim, é a característica mais utilizada para a construção de um modelo de representação para um documento. No entanto, termos multi-palavras podem perder um pouco de sentido quando isolados. Geralmente, para construir um modelo de representação baseado em palavras, seleciona-se somente aquelas que são mais representativas, eliminando o restante.
- **Termos**: termos podem sem compostos por uma única palavra ou por um conjunto de palavras que exprimem, por completo, a semântica que era desejada no texto. O processo de extração de termos é na maioria das vezes auxiliada por um dicionário de palavras, permitindo, assim, identificar termos que são compostos por mais de uma palavra.

1.4 TIPOS DE PROCESSOS DE MINERAÇÃO DE TEXTO

1.4.1 Análise estatística

Na análise estatística, os termos são valorados, basicamente, pela sua frequência de aparição na massa de dados, não importando a contextualização deste, como em que parágrafo está inserindo, que termos o antecedem ou que estão diretamente relacionados (CARRILHO, 2007).

Informações sobre contextualização, precedência ou sucessão de outros termos não são consideradas. Baseada no aprendizado estatístico, a principal vantagem desta abordagem é permitir a sua utilização em qualquer idioma (SOARES, 2008).

1.4.2 Análise semântica

"A análise semântica apoia-se no tratamento de textos conforme o ser humano faz, através do significado das palavras, de conhecimentos morfológicos, sintáticos, semânticos, pragmáticos e do contexto em geral" (CARRILHO, 2007).

Com o emprego de técnicas, estas baseadas no processo de PLN, capazes de avaliar e identificar a funcionalidade correta de um determinado termo em uma sentença, é possível obter a verdadeira importância do mesmo em seu contexto, possibilitando um ganho de qualidade dos resultados produzidos (SOARES, 2008).

O Quadro 1 resume as áreas de conhecimento mais envolvidas com os tipos de análise semântica e estatística. O Quadro 2 resume as principais características das duas abordagens.

Quadro 1 – As duas abordagens para a análise de textos e suas principais áreas de conhecimento.

Análise Estatística	Análise Semântica
Recuperação de Informação	Ciência Cognitiva
Estatística	Processamento de Linguagem Natural
Aprendizado de Máquina	Mineração de Dados
Inteligência Computacional	*Web Mining*
Mineração de Dados	
Web Mining	

Fonte: CARRILHO, 2007, p.15.

Quadro 2 – As principais características de cada uma das abordagens para a análise de textos.

Análise Estatística	Análise Semântica
Utilizável em qualquer idioma.	Necessita conhecimento específico do idioma que será objeto de análise.
Modelos com simples implementação e conhecidos na literatura.	Utiliza a informação semântica dos textos, tal como humanos.
Descarta qualquer valor semântico presente nos textos.	

Fonte: SOARES, 2008, p.34.

1.5 ÁREAS CORRELATAS À MINERAÇÃO DE TEXTOS

1.5.1 Ciência cognitiva

"A Ciência Cognitiva é normalmente definida como o estudo científico da mente ou da inteligência" (CARRILLHO, 2007). "Entender a mente humana requer numerosos métodos e teorias, por isso, desta área fazem parte a psicologia, a filosofia, a inteligência artificial, a neurociência e a linguística" (SOARES, 2008).

Algumas tecnologias podem demonstrar a importância da Ciência Cognitiva. Uma delas são as Redes Neurais, que são um exemplo de inteligência artificial conexionista, isto é, baseada na estrutura física e/ou biológica do cérebro humano, que adquirem conhecimento através da experiência.

1.5.2 Processamento de linguagem natural

"O Processamento de Linguagem Natural (PLN) é o campo da ciência que abrange um conjunto de métodos formais para analisar textos e gerar frases em um idioma humano" (ARANHA, 2007).

Segundo Carrilho (2007), o PLN tem papel fundamental na MT, sendo utilizado no estágio inicial da etapa pré-processamento, onde sua função é fornecer um primeiro nível de estruturação textual, com o reconhecimento de início e fim de sentenças, e classificação de palavras quanto à sua função sintática.

A Figura 4 mostra como é necessária a utilização de diferentes técnicas de PLN para melhorar o desempenho de cada uma separadamente.

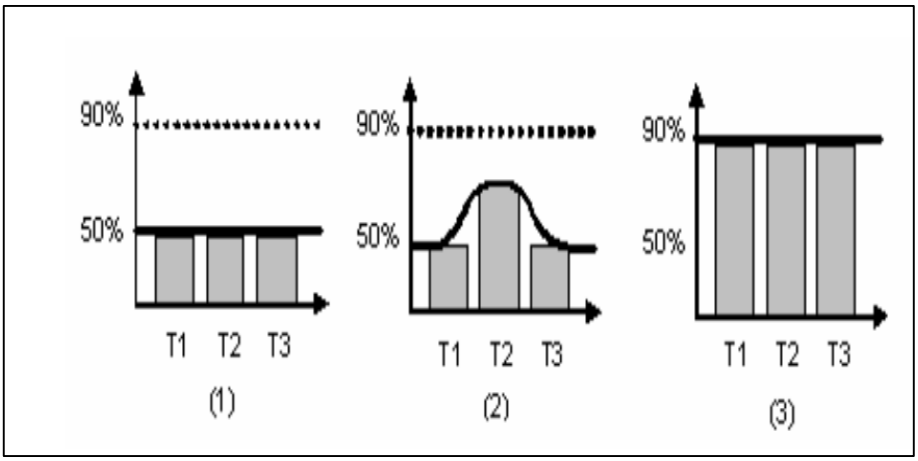

Figura 4: Os três gráficos (1), (2) e (3) mostram, de forma ilustrativa, a necessidade de aplicação de três tarefas diferentes de PLN T1, T2 e T3. T2 só consegue 90% de acerto de melhorar T1 e T3.

Fonte: ARANHA, 2007, p.60**.**

De uma forma geral, a eficiência de extração das entidades de um texto se comporta de forma exponencial à complexidade algorítmica. Com heurísticas simples, podemos alcançar um acerto de 80%. O grande desafio está em atingir valores acima dos 90%, observando que 100% é praticamente impossível.

1.5.3 Recuperação da informação

"Recuperação de Informação (RI) é uma área da computação que lida com o armazenamento e recuperação automática de documentos, que são objetos de dados, geralmente textos" (CARRILHO, 2007).

Sistemas de RI foram originalmente usados para gerenciar a explosão da informação na literatura científica, na segunda metade do século XX. Muitas universidades e bibliotecas públicas utilizam estes sistemas para prover acesso a livros, jornais, periódicos e outros documentos (SOARES, 2008).

Com a explosão da *web*, técnicas de recuperação da informação passaram a ser utilizadas em máquinas de buscas. A mais conhecida, atualmente, seria, dentre outras, o Google.

A MT tem, como característica, a utilização de algoritmos que requerem um grande esforço computacional, pois são aplicados em grandes coleções de documentos. Visando aperfeiçoar essa etapa da MT, a RI age diretamente na etapa de Indexação, onde são montadas estruturas de dados que permitem rápido acesso a documentos, através de palavras-chave, reduzindo drasticamente seu tempo de acesso.

Um sistema genérico de RI pode ser observado no esquema a seguir, ilustrado na Figura 5**:**

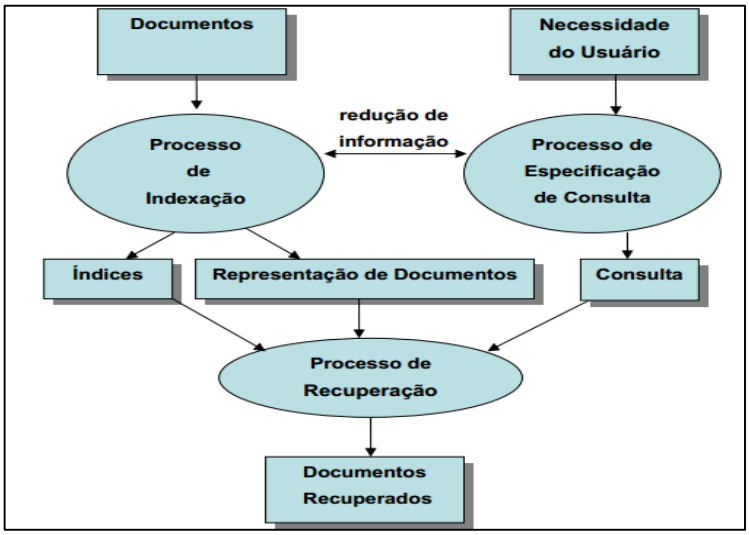

Figura 5: Componentes de um sistema de Recuperação de Informação.

Fonte: (CARRILHO, 2007, p.17)

O Processo de Indexação envolve a criação de estruturas de dados, associadas a parte textual dos documentos, como por exemplo, as listas invertidas.

O Processo de especificação da consulta geralmente é complexo. Pois existem uma distância semântica entre a necessidade real do usuário e o que ele expressa na consulta (CARRILHO, 2007).

O Processo de Recuperação consiste na resposta referente à consulta gerada pelo usuário, gerando uma listagem dos documentos. Os documentos são classificados de acordo com a similaridade entre o documento e a consulta.

1.5.4 Aprendizado de máquina

O Aprendizado de Máquina é uma subárea da Inteligência Artificial, onde o aprendizado envolve a generalização a partir da experiência: o desempenho deve melhorar não só na repetição da mesma tarefa, mas também em tarefas semelhantes no domínio (LUGER, 2004).

Soares (2008) ilustra, por meio de um esquema, o funcionamento de um modelo simples de aprendizagem de máquina, como pode ser observado na Figura 6:

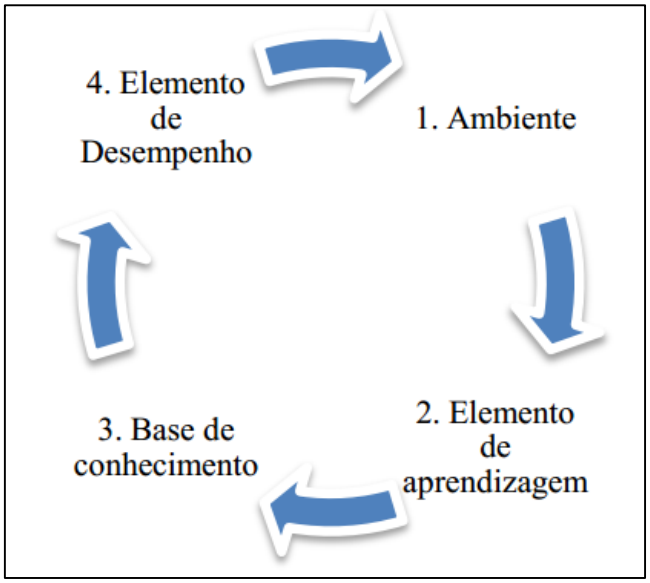

Figura 6: Modelo simples de aprendizagem de máquina.

Fonte: SOARES, 2008, p.37.

O ambiente fornece alguma informação para o elemento de aprendizagem. Esse, por sua vez, utiliza essa informação para aperfeiçoar a base de conhecimento. Por fim, o elemento de desempenho utiliza a base de conhecimento para a realização da tarefa requerida.

"Esse processo tem sido muito utilizado em tarefas de pré-processamento de textos como etiquetagem morfossintática, e no processo de classificação automática de textos" (CARRILHO, 2007).

1.5.5 - Estatística

Estatística é a ciência que, por meio de teorias probabilísticas, tem por objetivo a coleção, análise e interpretação de dados numéricos a respeito de fenômenos coletivos ou de massa, bem como a indução das leis a que tais fenômenos cabalmente obedecem e, ainda, a representação numérica e comparativa, em tabelas ou gráficos, dos resultados da análise desses (SPIEGEL, 2003).

Com base nesses fundamentos tem sido muito utilizado, no ramo da computação, um classificador probabilístico denominado *Naive Bayes*, apresentando ótimos resultados na categorização de documentos, onde seu teorema é ilustrado na Figura 7.

Teorema de Bayes	$$P(A\|B) = P(B\|A) \times \frac{P(A)}{P(B)}$$

Figura 7: Teorema de *Naive Bayes.*

Fonte: SOARES, 2008, p.39.

Outras tarefas na MT também são baseadas na estatística, como por exemplo, a seleção de amostra de dados, cálculos de aproximações, taxas de erros, médias e desvios, bem

como as validações de hipóteses e conhecimentos adquiridos ao final do processo de mineração (SOARES, 2008).

1.5.6 Mineração de dados

A Mineração de Dados é a principal etapa do processo de KDD, e é nela que ocorre a busca efetiva por conhecimentos novos e úteis, efetuando a aplicação de algoritmos sobre a base de dados, requerendo muitas vezes múltiplas interações (GOLDSCHMIDT & SOARES, 2008). “Ela provê um método automático para a descoberta de padrões em dados, sem a tendenciosidade e a limitação de uma análise baseada meramente na intuição humana” (BRAGA, 2005).

A área de mineração de dados contribuiu bastante para a de textos, em especial no que se refere à separação e estruturação do processo em etapas, assim como dos algoritmos e soluções utilizadas (CARRILHO, 2007).

2 ETAPAS DA MINERAÇÃO DE TEXTO

A tecnologia de mineração de texto é composta por um conjunto de processos que buscam, tratam e analisam grandes quantidades de textos. A seguir, uma explicação detalhada de cada etapa.

2.1 COLETA DE INFORMAÇÕES

"O processo de mineração de textos visa a busca de padrões em grandes coleções de arquivos textuais, que podem conter milhões de documentos, estas coleções são denominadas *Corpus ou Corpora"* (OLIVEIRA, 2009).

Segundo Feldman e Sanger (2007), a coleção de documentos que compõem o *corpus* pode ser classificada de duas maneiras: estática, onde o estado inicial da coletânea de textos permanece inalterado, ou dinâmica, que se caracteriza pela inclusão de novos documentos ou a atualização dos mesmos.

Existem, basicamente, três formas de obter material de qualidade para a formação de um *corpus* que sirva de alicerce para a aquisição de conhecimento, ressaltando que para uma boa formação do *corpus*, os documentos devem estar de acordo com o objetivo da extração de conhecimentos, uma vez que documentos que não incorporem informações relevantes podem prejudicar o processo (CARRILHO, 2007).

- Documentos em um disco rígido;
- tabelas de um banco de dados;
- ambiente *Web.*

A primeira opção para se conseguir material para um *corpus* é através de documentos alocados em um disco rígido. São documentos do nosso cotidiano, como arquivos do MS *Word*, Broffice, PDF (*Portable Format Document*), entre outros. O único cuidado a se tomar em uma coleta de dados em um disco rígido é com arquivos binários do sistema operacional, que podem vir a ser incorporados acidentalmente à coleção.

Outra forma de se obter uma boa base textual são os campos de armazenamentos de texto livre de um banco de dados, elementos geralmente do tipo *string*. Utilizar elementos tipo *string* exige um cuidado especial, uma vez que os campos podem estar localizados em várias tabelas e em diversos bancos de dados, tornando assim, essencial a implantação de um DW (*Data Warehouse*). A implantação de um DW tem por objetivo unificar as bases de dados, buscando tornar os dados mais sólidos. Outro aspecto a se observar nesse tipo de coleta é a execução de um pré-processamentos dos dados adquiridos. Essa "purificação" serve para garantir maior qualidade da informação.

A última forma de se obter dados para o *corpus* é pela internet, tendo como maior desafio a diversidade das fontes disponíveis, tais como: artigos de revistas, planilhas, documentos, redes sociais, anúncios, além da própria estrutura das páginas *Web*, o HTML (*Hypertext Markup Language* ou Linguagem de Marcação de Hipertexto) que se encontra em modo texto. A complexidade é um grande obstáculo à forma como se percorre os diferentes campos da internet. Assim se mostrou um problema em potencial, que seria extremamente custoso e demorado percorrer de forma manual página por página selecionando o material a ser acrescentando na base de dados. Portanto, para esse tipo de ambiente adotou-se o uso de ferramentas de apoio, motores de busca baseados em robôs que são denominados *crawlers* (CARRILHO, 2007).

2.1.1 Crawlers

Com a grande diversidade e a imensa quantidade de material disponível atualmente, proporcionada pelo intenso uso de computadores, a utilização de ferramentas que facilitem a coleta de informação tornou-se imprescindível.

Denomina-se *crawler* o software responsável por coletar dados em uma determinada rede, tornando-se mais populares na internet com os nomes de *webcrawler*, *webspy*, *webrobot*, entre outros. Segundo Heaton (2002) "um *crawler*, também conhecido como *web spider* ou *web robot*, é um robô que percorre a *web* de forma automática e metódica". Já Carrilho (2007) define *crawler* como:

> *Crawler* ou *Webcrawler* é o nome dado aos robôs especializados em navegar na *internet*, de forma autônoma e exploratória, com o objetivo de realizar a coleta

> automática de documentos. Para tanto, é necessário que as páginas HTML sejam interpretadas de forma correta, com a identificação de *hiperlinks*, seguido de visitação, conforme o ser humano realiza.

O processo de funcionamento de um *crawler* é bem simples, porém frágil, uma vez que envolve a interação com centenas de milhares de servidores *web* e servidores de nomes diferentes, que são todos fora do controle do sistema (ARANHA, 2007).

Inicia-se com a inserção de um site raiz denominado semente, *seed* ou *URL* (*Uniform Resource Locator)*, em português, Localizador Padrão de Recursos, que nada mais é o endereço de um recurso disponível em uma rede interna (Intranet) ou externa (Internet). Ao adentrar as *urls* o *crawler* começa a identificar os *hiperlinks* contidos no site e os armazena em uma pilha de endereços, onde os visitará posteriormente executando o mesmo processo, que pode ser visto na Figura 8.

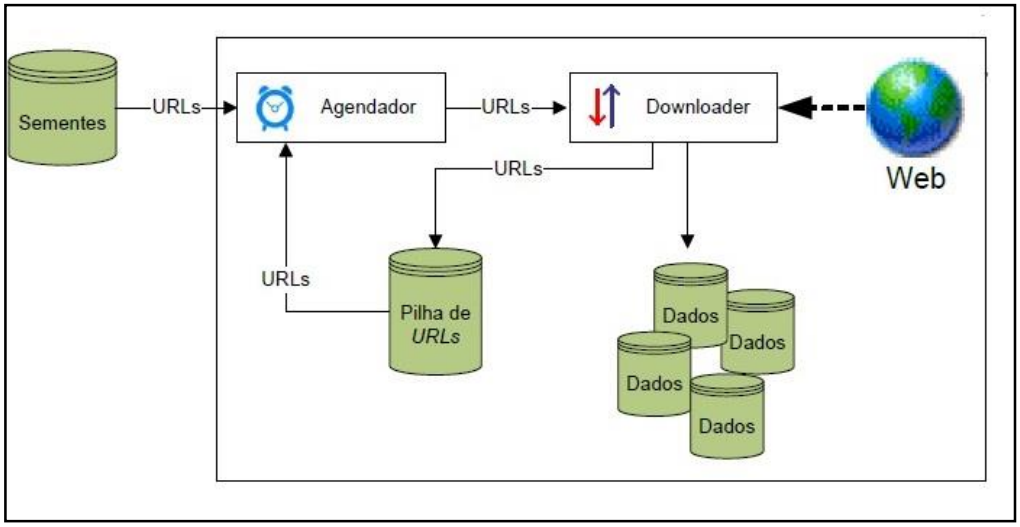

Figura 8 – Funcionamento de um *crawler* simples.

Fonte: SOARES, 2008, p.73.

Esse tipo de *software* é utilizado no cenário de S*earch Engines*, no português, Motores de Busca. Como exemplo tem-se o Yahoo Search, Qwiki, Microsoft Bing e o Google, que utilizam vários desses robôs de forma distribuída para varrerem a internet, coletando e armazenando dados que serão indexados para, posteriormente, serem usados no processo de busca. Porém, os *crawlers* também podem ser utilizados em tarefas mais simples, como executar manutenções em sites, verificando de quebra de *links* (SOARES, 2008).

2.1.2 Crawlers Focados

Crawlers focados são *softwares* que percorrem a *web* baseados em algum tipo de heurística, tornando, assim, a busca muito rápida e eficaz. Diferentemente do *crawler* comum, que apenas percorre de forma sequencial, o *crawler* focado utiliza-se de elementos adicionais para executar determinadas tarefas suplementares, como classificar a relevância das páginas e a forma de escalonamento dos *links* a serem visitados. Definir o que é ou não relevante, constitui a principal tarefa de um classificador, sendo elemento obrigatório de qualquer *crawler* focado (SOARES, 2008). A seguir, uma comparação entre *crawler* focado e *crawler* comum.

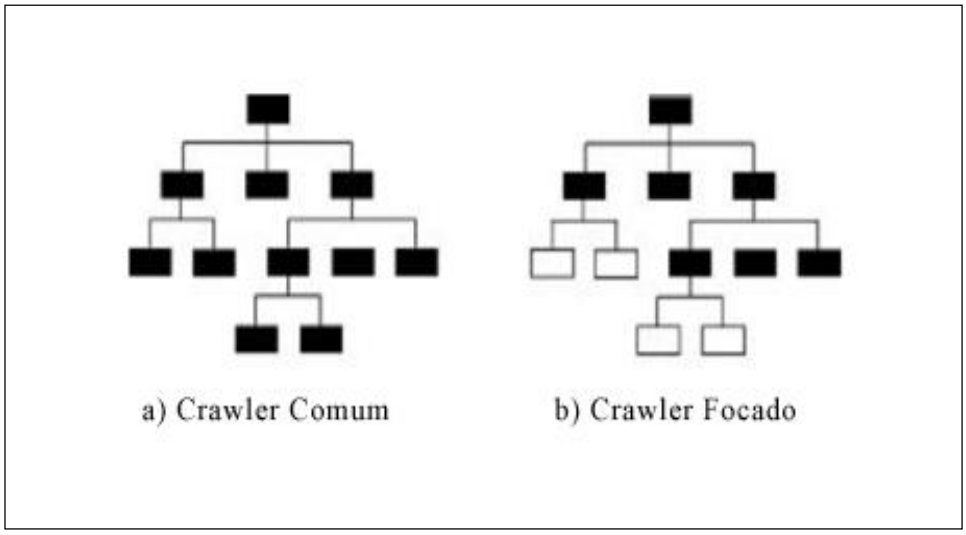

Figura 9 – Comparação entre *crawler* focado e *crawler* comum.
Fonte: SYED & BURKI 2011, p.2.

Como podemos observar na Figura 9, o *crawler* comum (a), faz a varredura independente da relevância das páginas que ele encontra. O que não acontece com o *crawler* focado (b), que descarta os endereços que não tem relevância para a consulta.

Para se obter o assunto desejado e diminuir os ruídos junto à sua coleção de textos, o *crawler* utiliza duas táticas, conforme texto a seguir:

> Um crawler focado pode contar com dois tipos de algoritmos ou até mesmo com alguma forma de combinação das duas ideias. São elas: *Web Analisys*, que julga

> relevância e qualidade das páginas apontadas por uma *URL* alvo; e *Web Search*, que determina a melhor ordem em que as *URL* alvo serão visitadas (SOARES, 2008).

A *Web Analisys*, faz um levantamento da relevância e qualidade das páginas, esse estudo pode ser realizado por duas vertentes, que são a análise do conteúdo da página *Content-Based* e análise das ligações dos links contidos na página *Link-Based*.

O algoritmo *Content-Based* analisa o teor de uma página utilizando técnicas de mineração de texto e indexação com análise textual, extraindo palavras chaves da página e confrontando-as com palavras de uma lista de termos representativos, por exemplo. E, a partir desta, é feita uma análise para se definir se o conteúdo é satisfatório ou não. Já, análise baseada nas ligações entre as páginas *Link-Based*, fundamenta-se na utilização de links para fundar critérios de relevância, levando em consideração que as ligações de uma página que foi inserida pelo autor, tenham alguma similaridade com o conteúdo da mesma. Como exemplo, podem ser consideradas as citações de uma monografia, onde o autor insere ligações com outras obras com conteúdo análogo ou complementar.

A *Web Search* visa sempre definir a melhor ordem em que as páginas serão visitadas. Para isso, visa-se não estender muito a busca além da *url* semente, já que quanto mais longe da semente, mais ruídos são adicionados ao *corpus*.

Para determinar a melhor forma de percorrer os grafos, a estratégia entra no cenário dos *crawlers* focados. Um bom exemplo é o algoritmo de *Breadth-first*, uma fórmula simples que não utiliza nenhuma heurística, simplesmente executa visitas as *url's* tanto em largura quanto em profundidade. Porém segundo Ardö (2006) esse método não obtém bons resultados na construção de grandes coleções, pois à medida que se coleta um grande número de paginas *web* a pesquisa começa há perder um pouco o foco. Poderá ocorrer uma grande quantidade de ruídos no *corpus* final, no entanto, com a combinação de *urls* bem selecionadas torna-se então muito eficaz em varrições em pequena escala.

Já para buscas complexas podem ser utilizadas estratégias mais elaboradas, que utilizam heurística computacional. Como exemplo, o *Best-First*, que emprega uma tática baseada em custo para classificar as *urls*, fazendo com que as mais promissoras fiquem no começo da fila, e as menos promissoras são colocadas no final da fila, sendo raramente visitadas. Mostra-se, assim, mais eficaz do que o algoritmo *Breadth-first*, já que caminha sempre em direção às páginas mais propícias e evita visitar páginas irrelevantes. Entretanto, o *Best-First* apresenta falhas. Por ser um algoritmo de busca local, só efetua sondagens a

vizinhos dos nós já visitados, podendo não adicionar ao *corpus* páginas importantes que não tenha nenhuma ligação (*link*) com aquela comunidade na *web* (Ardö, 2006).

Visando ter melhor aproveitamento nessas práticas, algumas técnicas têm, como objetivo, fugir de uma comunidade *web* para obtenção de uma maior abrangência de páginas. O *tunelling* ou tunelamento é uma dessas técnicas, visitando páginas não relevantes (sem precisar armazená-las), o algoritmo tenta escapar do escopo de varredura e alcançar outras comunidades.

Outras práticas empregadas para adentrar a diferentes comunidades *web* com conteúdo semelhante, utilizam métricas de co-citação, que diz que uma página tem relação com outra se ambas forem citadas por uma mesma terceira página. Pode-se observar claramente, de forma simplificada, as duas técnicas na Figura 10.

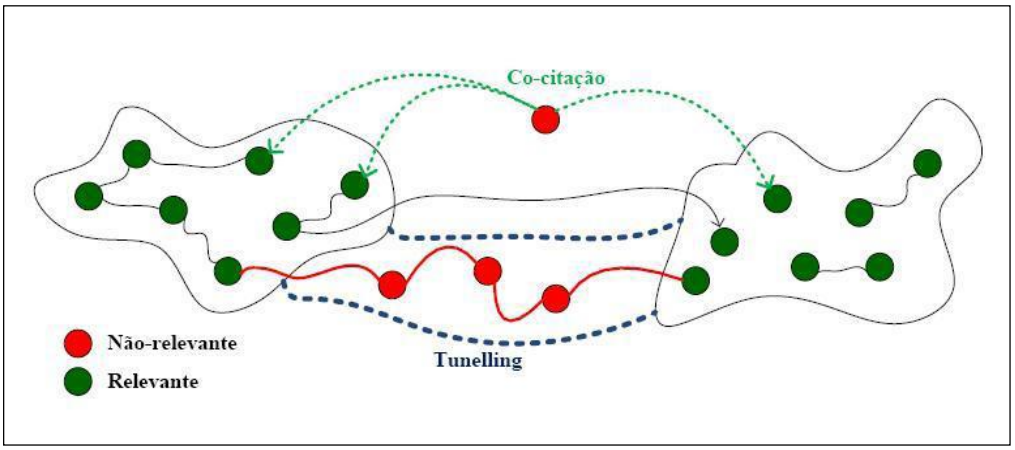

Figura 10 – Relacionamentos de co-citação e tunelamento em uma comunidade *Web.*

Fonte: SOARES, 2008, p.88.

2.2 PRÉ-PROCESSAMENTO

A etapa de pré-processamento é aplicada logo após a etapa de coleta, visando modificar os dados recolhidos e apresentá-los de uma forma estruturada, para a aplicação de algoritmos em etapas futuras. Para Aranha (2007, p.42) “a etapa de pré-processamento tem por finalidade melhorar a qualidade dos dados já disponíveis e organizá-los”.

Para chegar ao objetivo, a etapa de pré-processamento dispõe de várias técnicas, que vão desde dividir o texto em palavras, até classificá-las conforme a classe gramatical. Devido a não ter uma procedimento que tenha um resultado satisfatório em todos os domínios, essa etapa é considerada um dos processos mais custosos da tecnologia de mineração de texto, já que para chegar ao um resultado satisfatório é necessário se aplicar um conjunto de métodos (CARRILHO, 2007).

Ao final do processo, o que se espera, em geral, é a obtenção de uma tabela com a representação de atributos/valor, como é possível observar Quadro 3.

Quadro 3 – Representação atributo-valor obtida a partir da etapa de Pré-processamento.

	Atrib1	**...**	**AtribN**
Doc1	V11	...	V1N
...	....	....	....
Docm	Vm1	...	Vmn

Fonte: CARRILHO, 2007, p. 30.

2.2.1 Tokenização

Tokenization, atomização ou tokenização é o primeiro processo da etapa de pré-processamento. Esta etapa tem, como objetivo, extrair as unidades mínimas dos documentos textuais, unidades que são denominadas como *tokens*. Os *tokens* geralmente são formados por palavras do texto, caracteres e símbolos de pontuação. Porém, existem ocasiões onde é necessário o agrupamento de dois ou mais *tokens*, como por exemplo, o nome composto de uma entidade, onde só existe semântica no conjunto das palavras. Uma sequência ou conjunto de *tokens*, por sua vez, é denominado *tokenstream*. Neste caso, o único caractere descartado é o espaço em branco.

Um exemplo pode ser observado na sentença a seguir: “Amanhã se não chover, irei jogar futebol!”. Separando em *tokens*, essa sentença ficaria da seguinte forma: [Amanhã] [se] [não] [chover] [,] [irei] [jogar] [futebol] [!], onde cada colchete simula um *token* da frase.

Porém, “o que parece uma tarefa extremamente fácil para o ser humano, pode se tornar um desafio bastante complexo para uma máquina” (CARRILHO, 2007).Os caracteres delimitadores armazenados em grandes quantidades, na maioria das vezes são considerados irrelevantes à apresentação do conteúdo, e são descartados. Entretanto, o grande problema, segundo Carrilho (2007), se encontra exatamente em caracteres delimitadores, pois estes podem assumir papéis importantes. Por exemplo, o ‘ponto’ pode determinar o fim de uma sentença, e também pode ser utilizado em abreviações e números.

É possível refinar o resultado da tokenização. Para isso, pode-se adicionar regras de formatação que, com o auxílio de dicionários, identificam e tratam casos específicos (CARILLHO, 2007).

Após a geração simples dos *tokens*, é feita a identificação de abreviações. Com a ajuda de um dicionário pré-estabelecido, podem ser identificadas abreviações com uma ou mais palavras, e com ou sem pontos. Com a mesma técnica, pode se identificar símbolos da internet como *urls* de *websites*, endereços IPs, símbolos referentes a *e-mail* etc, como por exemplo: http://www.unifran.com.br. Pode-se perceber que ao separar os *tokens*, perde-se a referência do site em questão. Palavras combinadas são palavras que separadas não podem ter o mesmo sentido de que quando juntas. Por exemplo, a palavra composta Coca-Cola, se seus termos forem separados, as palavras não terão o mesmo sentido de quando lidas em sequência. No Quadro 4 a seguir, podemos observar um exemplo de geração de *tokens*.

Quadro 4 – Exemplificação do resultado da execução de um subsistema de *Tokenization* que se baseia em dicionários pré-estabelecidos e regras de formação.

Descrição	Tokens
Primeiramente, o fluxo de texto é apresentando ao subsistema de *Tokenization*.	A Casa & Lar se mudou. Agora, atendemos na Av. Dom Casmurro, nº 200. Você também pode comprar pelo nosso endereço na Internet: http://www.casaelar.com.br. Comprando por lá, você tem até R$ 100,00 de desconto em tubos e caixas d'água!
Em seguida são gerados os *tokens* preliminares, com a observação de espaços em branco e delimitadores.	[A][Casa][&][Lar][se][mudou][.][Agora][,] [atendemos] [na] [Av][.] [Dom] [Casmurro][,] [n][º] [200][.] [Você] [também][pode] [comprar] [pelo] [nosso] [endereço] [na] [Internet][:] [http][:][/][/][www][.][casaelar][.][com][.][br][.] [Comprando] [por] [lá][,] [você] [tem] [até] [R$] [100][,][00] [de] [desconto][em][tubos][e][caixas][d][‘][água][!]
Logo após, são identificadas as abreviações (em verde), as palavras combinadas (em amarelo) e os símbolos de Internet (em azul).	[A][Casa&Lar][se][mudou][.][Agora][,][atendemos] [na] [Av.] [Dom] [Casmurro][,] [nº] [200][.] [Você] [também][pode] [comprar] [pelo][nosso] [endereço] [na][Internet][:] [http://www.casaelar.com.br][.][Comprando][por] [lá][,] [você] [tem] [até] [R$] [100][,][00] [de] [desconto][em][tubos][e][caixas][d][‘][água][!]
Por último são identificados os números e os *tokens* multi-vocabulares. Esta é a lista final que será retornada pelo subsistema *Tokenization* de acordo com o exemplo de entrada.	[A][Casa&Lar][se][mudou][.][Agora][,] [atendemos] [na] [Av. Dom Casmurro][,] [nº 200][.] [Você] [também][pode] [comprar] [pelo] [nosso] [endereço] [na] [Internet][:] [http://www.casaelar.com.br] [.] [Comprando] [por] [lá][,] [você] [tem] [até] [R$ 100,00] [de] [desconto][em][tubos][e][caixas d'água][!]

Fonte: CARRILHO, 2007, p. 33.

2.2.2 Redução do léxico

Um dos maiores problemas da mineração de texto, segundo Aranha (2007), é lidar com o elevado número de dimensões existentes, levando em conta que cada termo representa uma dimensão.

Existem varias técnicas que são aplicadas para realizar a redução do léxico, métodos que buscam extrair apenas os termos realmente importantes e que traduzem o verdadeiro sentido do texto. A seguir, são apresentadas as mais citadas na literatura.

2.2.3 Seleção de características

A seleção de características tem como objetivo definir, através de aplicação de algoritmos e métricas, o conjunto mais discriminante de *n* conjuntos de características, visando, então, reduzir o espaço inicial (CARRILHO, 2007).

O maior cuidado a ser tomado quando se aplica procedimentos para redução do léxico é obter uma perda mínima de informação para que não afete a semântica do texto. Para isso, usam-se técnicas de estatística para calcular a importância de determinado termo para o léxico. A frequência de documentos é uma destas práticas. Utiliza um sistema bem simples, que consiste em computar o número de documentos que determinado termo aparece, e elimina termos que têm baixa frequência. Segundo Carrilho (2007) “a suposição é que termos raros não são significativos para a discriminação entre classes de texto ou que, pelo menos, esta eliminação não impacte no desempenho global”.

2.2.4 Remoção de *Stopwords*

Esta fase envolve a remoção de termos que não possuem valor semântico para as próximas etapas do processamento textual. Artigos, preposições, pronomes e conjunções são termos úteis apenas para a compreensão geral do texto. Segundo Bastos (2006), “estas palavras são classificadas como *Stopwords* e fazem parte do que, na literatura de mineração

de texto, ficou conhecido como *Stoplist*". Comumente, palavras com uma alta frequência de aparição nos documentos e com uma baixa relevância semântica serão desconsideradas na análise textual.

O processo de remoção de *Stopwords* reduz, consideravelmente, o tamanho do léxico, tendo assim um aumento de desempenho. "Uma *Stoplist* bem elaborada permite a eliminação de muitos termos irrelevantes, tornando mais eficiente o resultado obtido pelo processo de Mineração de Texto" (SOARES, 2008).

Uma *Stoplist* pode ser definida de duas maneiras: manual, que exige um especialista no assunto para indicar dos termos que serão removidos, ou de maneira automática, através da frequência de aparição das palavras no léxico.

No Quadro 5 é possível observar um exemplo de uma pequena *Stoplist*.

Quadro 5 – Identificação e remoção de *Stopwords*.

Stoplist				**Texto**
A	seu	De	Um	~~[A]~~[Casa&Lar]~~[se]~~[mudou]~~[.]~~[Agora]~~[,]~~
O	deve	do	uma	[atendemos] ~~[na]~~ [Av. Dom Casmurro]~~[,]~~ [nº
pelo	sua	da	sobre	200]~~[.]~~ [Você] ~~[também]~~[pode]—[comprar]
por	nosso	também	são	~~[pelo]~~ [nosso] [endereço] ~~[na]~~ [Internet]~~[.]~~
em	nossa	se	cada	[http://www.casaelar.com.br] ~~[.]~~
na	.	comigo	isso	[Comprando] ~~[por]~~ ~~[lá]~~~~[,]~~ [você] [tem] [até]
no	!	pela		[R$ 100,00] ~~[de]~~
como	;	?		[desconto]~~[em]~~[tubos]~~[e]~~[caixas d'água]~~[!]~~
lá	,	só		

Fonte: CARRILHO, 2007, p. 39.

2.2.5 Normalização

A normalização é uma técnica de redução de léxico, que identifica e agrupa palavras que podem ter algum tipo de relação. Existem vários métodos utilizados para realizar tal agrupamento, que vão da análise da constituição da palavra, à identificação de sinônimos.

> A ideia é esquivar das várias formas de representação de uma palavra associada a um mesmo conceito. Por exemplo, do conceito de "objeto físico que consiste em um

> número de páginas atadas juntamente" temos a palavra "livro" com as seguintes representações "livro" e "livros" (plural). O processo de normalização propõe que essas duas formas sejam agrupadas em apenas uma, indicando que, para a busca, elas têm o mesmo significado (Aranha, 2007).

De acordo com Carrilho (2007), dependendo do tamanho da massa textual e do que se quer obter na saída, a técnica de normalização pode trazer uma melhora significativa no processo de mineração de texto, tendo um grande beneficiamento em sistemas de classificação que utilizam práticas de estatísticas como base. Dentre as várias técnicas utilizadas para normalização, a seguir uma breve definição das principais.

- **Lematização**: substituem-se as variadas formas de representação de uma palavra pela sua forma primitiva, assim, palavras como "música", "músicas", "musical", todos esses tokens referenciam a palavra "música". Segundo Carrilho (2007), essa técnica tem a vantagem de manter a estrutura que mantém os sentidos das palavras, ao contrário do método de *stemming.*
- ***Stemming***: o processo de s*temming* busca reduzir cada *token* do léxico para sua forma raiz, eliminando variações como plural e tempos verbais. Vários métodos são utilizados para aplicar o processo de s*temming*, cada um apresenta uma característica diferente.
- **Método de *Stemmer S***: método bastante simples, que busca apenas eliminar os principais sufixos finais de palavras do inglês, como por exemplo: ies, es e s (com exceções). Método conservador e que raramente surpreende o usuário negativamente (LOPES, 2007).
- **Método de Porter**: este método tem como função analisar as palavras do texto e as converter para seu radical, ou seja, ele faz com que palavras semelhantes referenciem uma palavra formada por um radical comum. Assim, palavras como música, músicas, músico e musical apontam para o mesmo radical, music.
- **Identificação de Sinônimos, Hierarquias e Relacionamentos Associativos:** Esta técnica visa atribuir a diferentes termos que tenham o mesmo significado uma palavra que as represente. Geralmente este mapeamento é feito com a ajuda de um *thesaurus.* Um thesaurus é definido como um repositório de mapeamentos entre termos variantes – sinônimos, abreviações, acrônimos e ortografias alternativas – para um termo preferido único para cada conceito (CARRILHO, 2007, p.41).

2.3 INDEXAÇÃO

Após a etapa de pré-processamento, os textos antes em linguagem natural, agora possuem uma representação estruturada. Porém a recuperação de informação dessas estruturas demanda grande esforço computacional, já que para realizar uma simples consulta a máquina teria que analisar documento a documento, percorrendo assim toda coleção. De acordo com Soares (2008), a etapa de indexação é responsável por criar uma estruturação baseada em índices, visando permitir que uma consulta seja efetuada sem que seja necessário analisar toda base de dados.

Devido ao surgimento dos grandes motores de busca na internet como o Google, Yahoo! Search, Microsoft Bing, entre outros, as técnicas de recuperação de informação em documentos tiveram grandes avanços. Existem vários modelos para representação de documentos na literatura de RI, porém segundo Carrilho (2007), o que mais contribui para os estudos de mineração de texto é o Modelo de Espaço Vetorial, em inglês, *Vector Space Model,* que utiliza uma representação geométrica, onde cada documento é um ponto em um espaço Euclidiano.

Na literatura de mineração de texto é comum a apresentação deste modelo pelo nome de Saco de Palavras ou *Bag-of-Words,* onde o documento é visto como um *container* de tokens. Com uma abordagem puramente estatística, esse modelo vem apresentando bons resultados em pesquisas da área (CARRILHO, 2007).

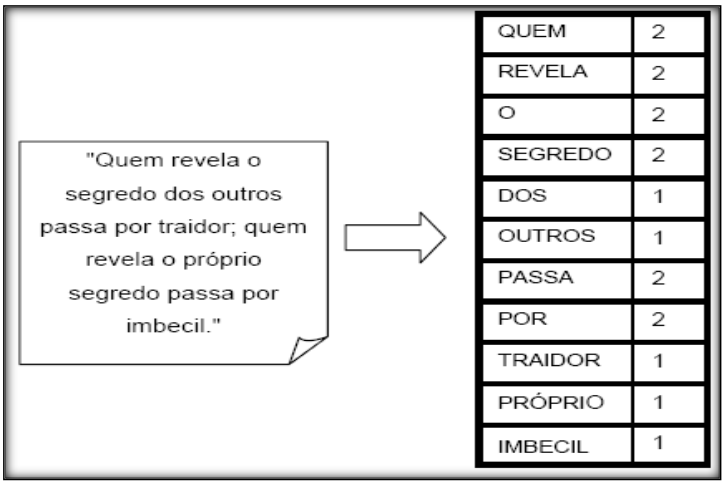

Figura 11 – Representação do modelo *Bag-of-Words*

Fonte: CARRILHO, 2007, p. 49.

2.3.1 Listas Invertidas

O conceito de Listas Invertidas vem com a proposta de busca diferente da convencional. Ao invés de ter os documentos apontando para tokens, tem-se tokens em uma estrutura de índice apontando para os documentos onde estão contidos. Basicamente, a estrutura permite que um único termo aponte para vários documentos (ARANHA, 2007). Pode-se observar, na Figura 12, apresentada por Carrilho (2007), uma representação do conceito de listas invertidas.

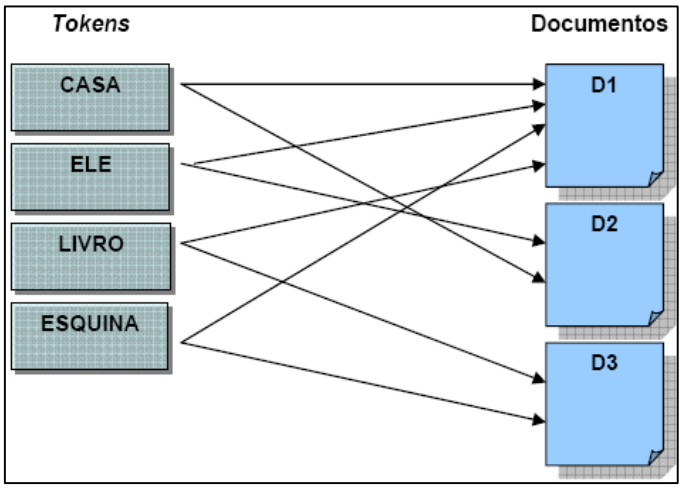

Figura 12 – Representação de uma lista invertida

Fonte: CARRILHO, 2007, p. 52.

Com a utilização da técnica de lista invertida pode-se otimizar a velocidade de processamento de uma consulta , pois o termo a ser consultado é acessado diretamente pelo índice, realizando uma busca direta e evitando uma consulta sequencial, documento por documento.

Segundo Carrilho (2007), existem dos tipos de índices. O primeiro, é o tipo comum, onde cada termo referencia os documentos onde se encontra. Quanto ao segundo tipo, os termos não só apontam para os documentos que os armazenam, mas também para a posição onde eles se encontram dentro do arquivo, permitindo assim a elaboração de consultas mais complexas.

2.3.2 Indexação Temática

No processo de indexação temática, as palavras encontradas nos textos são repassadas para um dicionário chamado *Thesaurus.* O *Thesaurus* é um conjunto de termos que define um vocabulário, e é montado usando relacionamentos (LOPES, 2004). A função do *Thesaurus* é utilizar um termo relacionado para indexar palavras com a mesma semântica. Por exemplo, veículo, automóvel e carro poderiam ser relacionados com apenas um termo, este denominado preferido, que referencia um conjunto de termos sinônimos chamados não-preferenciais. A Figura 13 ilustra o funcionamento de um dicionário de *Thesaurus*.

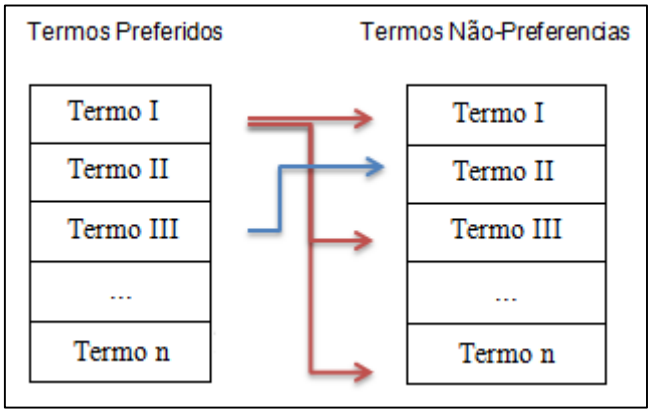

Figura 13 – Funcionamento básico de um dicionário *Thesaurus.*

Fonte: Os autores.

A utilização desta técnica, além de tornar o índice mais compacto, permite a localização de documentos grafados de forma diferente, mas que apresentam mesmo valor semântico (SOARES, 2008).

2.4 MINERAÇÃO DE TEXTO

Na etapa de mineração são escolhidos os algoritmos que serão aplicados para extrair conhecimento útil das massas textuais, obtidas a partir dos processos anteriores.

Segundo Soares (2008), a escolha do algoritmo a ser aplicado nas bases de texto está diretamente relacionada com o objetivo da tarefa de mineração de texto, ou seja, primeiro deve-se decidir que tipo de informação se deseja obter com a coleção de textos. A partir de então, pode-se utilizar algoritmos provenientes de varias áreas de conhecimento, tais como Estatística, Aprendizado de Máquina, Redes Neurais, Banco de Dados, entre outros (ARANHA, 2007).

Na fase de mineração há outros detalhes que devem ser considerados, como por exemplo, se o conhecimento obtido tem a necessidade ou não de ser facilmente interpretado, ou a urgência do processo, pois Soares (2008) afirma que algumas alternativas, embora apresentem bom resultados, devem ser descartadas, pelo fato de precisarem de um alto tempo de processamento para o treinamento das mesmas.

A seguir, serão abordadas, com maiores detalhes, as principais tarefas da etapa de mineração de texto e seus principais algoritmos.

2.4.1 Categorização de Texto

A tarefa de categorização tem como objetivo a organização automática da coleção de texto, tendo trabalho de identificar os tópicos principais de um documento e associar determinado documento em uma ou mais categorias pré-definidas (CARRILHO, 2007). Os documentos são organizados, por exemplo, os livros de uma biblioteca. Livros que possuem o mesmo tema ou que têm algo em comum, tendem a ficar mais próximos uns dos outros. Da mesma forma funciona a categorização de textos, ou seja, documentos que compartilham de temas iguais ou parecidos são adicionados em uma mesma categoria, facilitando o acesso a determinado conteúdo.

De acordo com Lopes (2004), as categorias podem ser criadas de duas maneiras. A primeira baseia-se na frequência das palavras especificadas em cada domínio. A segunda é a partir do treinamento de uma ferramenta de categorização. Para o treinamento um conjunto de documentos exemplares que representam cada categoria é inserido na ferramenta de categorização. A partir da análise linguística, cria-se uma assinatura estatística de cada categoria. Por fim essas assinaturas são aplicadas sobre outros documentos a fim de encontrar os mais parecidos com cada assinatura.

A Figura 13 mostra o cenário de criação de categoria com uma ferramenta de categorização:

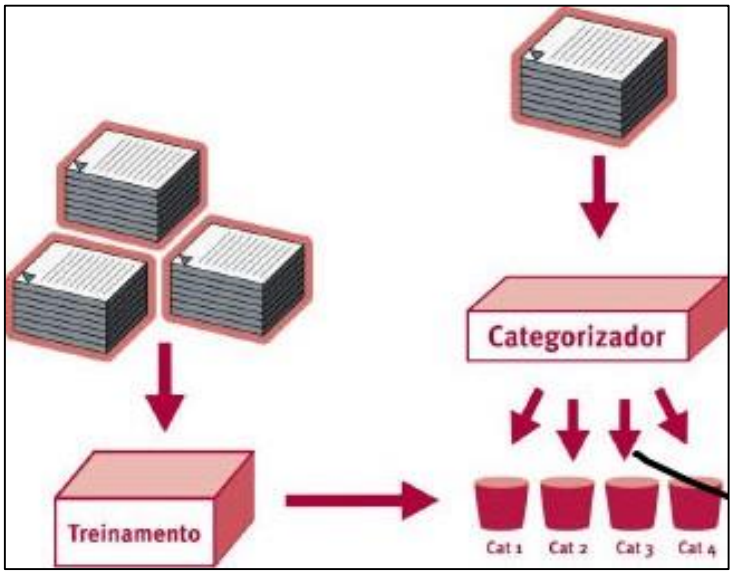

Figura 14 – Processo de categorização.

Fonte: LOPES, 2004, p. 41.

Os algoritmos de categorização se dividem em duas categorias gerais. A primeira categoria se baseia em algoritmos de aprendizado de máquina. É possível encontrar algoritmos como árvore de decisão, classificadores probabilísticos, redes neurais, SVM (*Support Vector Machines*), entre outros (CARRILHO, 2007). Já, a segunda categoria contém algoritmos originários da área de recuperação de informação, tais como *feedback* de relevância, classificadores lineares, classificadores de conjunto de exemplos genéricos etc (LOPES, 2004).

2.4.2 Clusterização

Clusterização, *Clustering* ou agrupamento é a tarefa que determina uma organização, em grupos chamados *clusters*, para uma coleção de documentos (CARRILHO, 2007). Em outras palavras, os algoritmos de clusterização têm por finalidade agrupar textos que possuem algum grau de similaridade. Para calcular o grau de similaridade entre os documentos, normalmente usa-se métricas computadas em números. Portanto, é necessário

que o documento seja mapeado em forma vetorial. E com base nos vetores de características que são gerados, pode-se determinar qual o grau de semelhança entre dois documentos (CARRILHO, 2007).

> *Clustering* é uma técnica útil quando se quer agrupar documentos similares. O processo de *Clustering* básico funciona da seguinte forma: primeiro, uma descrição simplificada do documento é criada para cada texto que vai sendo adicionado aos clusters. A descrição é normalmente um vetor de características, ou uma lista de temas dominantes ou palavras-chave e uma medida de importância relativa de cada tema ou palavra-chave no documento (LOPES, 2004).

A tarefa de clusterização é considerada bastante complexa, devido à quantidade de parâmetros exigidos pelos algoritmos que a realizam (CARRILHO, 2007). A Figura 15 mostra a esquematização básica de um algoritmo de clusterização sobre uma coleção de texto, separando-os em subgrupos denominados *clusters*.

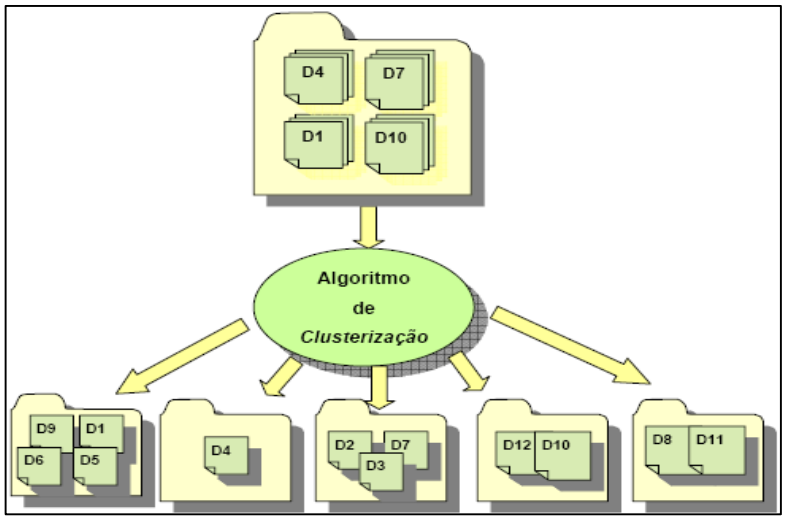

Figura 14 – Processo básico de clusterização.

Fonte: CARRILHO, 2007, p. 67.

Existem inúmeras técnicas de clusterização que podem ser utilizadas para agrupar documentos textuais. A seguir, uma breve descrição dos principais métodos presentes na literatura de mineração de texto.

- ***Clustering* Hierárquico:** a clusterização hierárquica é um método bastante complexo, onde pares de documentos são ligados continuamente até todos os documentos estarem interligados, formando uma árvore hierárquica chamada dendograma.

- ***Clustering K-means:*** a partir de um número fixo determinado K, a técnica de *Clustering K-means* gera um conjunto de K *clusters*, e distribui o conjunto de documentos textuais entre esses *clusters* com base na similaridade entre os vetores dos documentos e os centróides dos *clusters.* É nominado centroide o vetor médio de todos os vetores dos documentos contido em determinado *clusters.* Segundo Lopes (2004), acredita-se que os métodos de *clustering* hierárquicos produzem clusters de melhor qualidade que aqueles produzidos pelo K-means.
- ***Clustering* de Palavras:** em mineração de texto, a técnica de clusterização também pode ser aplicada para reduzir o número de dados de uma coleção de documentos, agrupando palavras similares e formando um *cluster* de tokens. Assim, o documento pode ser representado por um *cluster* e não mais por palavras individuais.

2.4.3 Sumarização

A tarefa de sumarização visa a redução da massa textual, com o objetivo de maximizar o desempenho no momento de busca de informação. Porém, essa tarefa tem como desafio a eliminação de dados, sem a perda dos significados-chave do texto.

- **Sumarização por Abstração:** este tipo de sumarização tenta criar resumos de forma automática, da mesma forma que o ser humano. Um pequeno documento é gerado a partir do entendimento do leitor, mantendo a ideia principal do texto original, porém, com novas palavras e sentenças.
- **Sumarização por Extração:** esta técnica concentra-se na criação de resumos através da seleção de sentenças e parágrafos principais e importantes, copiados inteiramente do texto original (CARRILHO, 2007). Esta técnica se baseia na importância relativa das palavras de um documento para realizar a extração da sentença do texto.

Segundo Carrilho (2007), a tarefa de sumarização utiliza-se de outras duas técnicas para concluir seu objetivo: classificação e clusterização, que foram abordadas em tópicos anteriores.

2.4.4 Extração de Informação

Extração de informação ou Extração de Características é a tarefa que visa obter dados estruturados a partir de dados não estruturados. Geralmente, a informação extraída é armazenada em tabelas, também chamadas de *templates*, como é ilustrado na Figura 16.

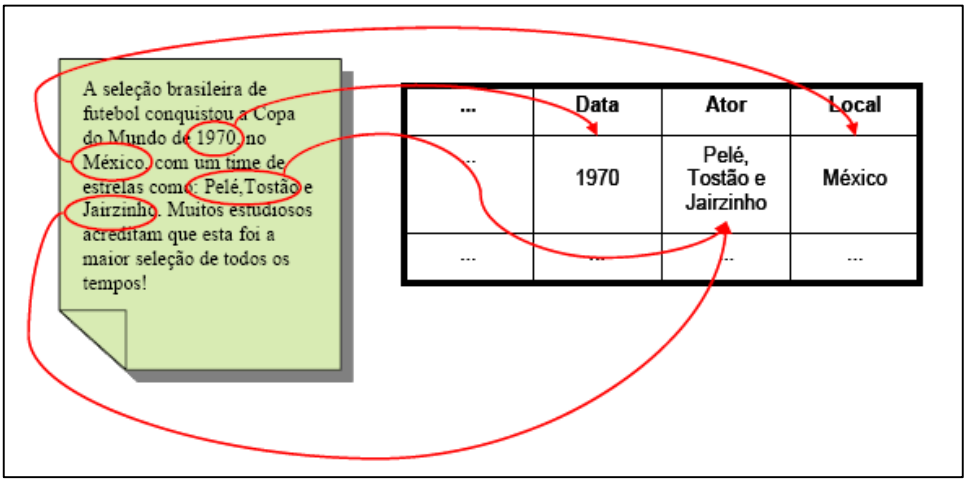

Figura 16 – Processo de Extração de Informação.

Fonte: CARRILHO, 2007, p. 71.

Com ao auxílio dos *template* gerados, pode-se, agora, aplicar, segundo Carrilho (2007), algoritmos clássicos de Data Mining já que, neste momento, os textos encontram-se em uma forma estruturada.

Sistemas de extração de informação utilizam variadas técnicas para realizar suas tarefas. Uma delas é o processamento de linguagem natural ou PLN, utilizada para extrair a informação necessária. Nessa área é muito comum a utilização de técnicas como autômatos finitos e expressões regulares. Outro método utilizado é a tarefa de classificação para associar *tokens*, classificando-os em classes distintas. Por exemplo, podemos classificar determinado *token* como um “numero de telefone” ou um “endereço” (CARRILHO, 2007).

2.5 ANÁLISE

Etapa de pós-processamento ou etapa de análise é o momento de se ponderar os resultados obtidos a partir da aplicação dos algoritmos de mineração, e concluir se o objetivo principal, que é a aquisição de conhecimento útil, foi atingido.

Segundo Carrilho (2007), existem diversas maneiras de se avaliar a mineração como um todo, seja de forma qualitativa ou quantitativa. Avaliações quantitativas são quando os resultados alcançados são avaliados por métricas baseadas em noções de relevância. Se um documento atender às necessidades de informação do usuário, ele é considerado relevante à solicitação do usuário (LOPES, 2004). O acompanhamento de um especialista no assunto deve incidir em todas as etapas do processo de mineração de texto, auxiliando e complementando informações e, na etapa de análise, o profissional é quem valida o conhecimento que foi obtido, realizando então uma avaliação qualitativa.

A apresentação da informação adquirida é uma questão muito importante, já que após o método de extração de conhecimento, a informação às vezes pode não parecer muito clara. “A introdução de gráficos, com noções de cores e distâncias, ajuda a entender o sentido de grandes e complexos conjuntos de dados, que não são facilmente manuseados” (CARRILHO, 2007).

Por fim, é importante lembrar que ao término da etapa de análise, se o resultado obtido não for satisfatório, pode-se optar por voltar a qualquer etapa anterior, com o intuito de corrigir falhas que originaram ruídos na informação obtida.

3 CLASSIFICADOR DE *TWEETS*

Neste capítulo será apresentada a fundamentação teórica e metodologia aplicada na implementação do protótipo de classificador de *tweets*.

3.1 ALGORITMO DE CLASSIFICAÇÃO

No processo de mineração de texto existem várias formas de se extrair conhecimento de uma massa textual. Neste trabalho será utilizado como técnica base de extração de conhecimento o método de classificação.

O objetivo do protótipo é classificar de forma automática os pequenos textos retirados da rede-social Twitter, denominados *tweets*, em duas categorias: positivos e negativos. Para a execução de determinada tarefa, foi abordada uma técnica de categorização que utiliza aprendizagem máquina, o *Naive Bayes*. O algoritmo de *Naive Bayes* é um modelo de aprendizagem supervisionada, ou seja, ele se baseia em uma base de dados já rotulada para aprender.

Para implementação do classificador descrito, será utilizada a linguagem Python e o kit de processamento de linguagem natural NLTK, que por sua vez, fornece o algoritmo de *Naive Bayes* para a aplicação.

Ao final do desenvolvimento o protótipo deverá ser capaz de capturar *tweets*, processa-los e classificar se seu conteúdo é positivo ou negativo. A seguir uma explicação mais detalhada das ferramentas e das técnicas abordadas na implementação.

3.2 LINGUAGEM DE PROGRAMAÇÃO PYTHON

Python é uma linguagem de programação de alto nível, projetada em 1991 por Guido van Rossum. É uma linguagem interpretada, de código-fonte aberto e está disponível para diversos sistemas operacionais. Entre suas principais características estão:

- Python é uma linguagem de tipagem forte e dinâmica;

- suporta múltiplos paradigmas de programação, tais como funcional, modular e orientado a objeto, este último contendo todos os conceitos teóricos, por exemplo, conceito de classes, herança, polimorfismo etc;
- o controle de um bloco de código em Python é feito apenas por endentação não há delimitadores como { e }.

Para um melhor entendimento sobre a linguagem Python segue a baixo um exemplo de um “Olá Mundo” em Python.

```
>>> # -*- coding: iso-8859-15 -*-
>>> texto = 'Olá Mundo'
>>> print(texto)
Olá Mundo
>>> print(type(texto))
<type 'str'>
>>>
```

Figura 17 – Programa “Olá Mundo” em Python.

Fonte: Os autores.

Na Figura 17 podemos observar algumas das características do Python, como por exemplo o uso do *encoding* na primeira linha, que é necessário para utilização de caracteres especiais, como acentos. Na segunda linha podemos observar a atribuição de uma *string* a uma variável e a tipagem dinâmica do Python, onde não é preciso indicar o tipo da variável “texto”. Na terceira e quarta linha mostram consecutivamente, o comando para exibir a variável e a sua exibição, e na quinta e sexta linha podemos observar o comando dado para exibir o tipo da variável e seu resultado, exibindo o resultado da tipagem dinâmica do Python, classificando a variável “texto” como uma *string*.

Em Python praticamente tudo é um objeto, ou seja, uma variável do tipo *string* não é apenas uma variável, mas sim um objeto do tipo *string*, como a maioria dos objetos ele também possui métodos próprios. Um dos pontos fortes do Python é a facilidade que a linguagem proporciona para manipular *strings*. Vejamos alguns exemplos de métodos muito utilizados quando se trabalha com texto.

Um método muito utilizado quando se trabalha com *strings* é o método *replace*, que tem como função substituir palavras da *string*, como mostra a Figura 18.

```
>>> Texto = 'Python é muito legal'
>>> print(Texto)
Python é muito legal
>>> Texto = Texto.replace('legal','fácil')
>>> print(Texto)
Python é muito fácil
```

Figura 18 – Método *replace* em Python.

Fonte: Os autores.

Outro método de objetos do tipo *string* é o método s*plit*, que é empregado para separar uma *string*. O método pode ser chamado de duas maneiras, sem parâmetro, separando a *string* nos espaços em branco ou com parâmetro, dividindo a *string* na palavra usada como parâmetro, como é exemplificado na Figura 19.

```
>>> Texto = 'Apenas uma string comum'
>>> Texto.split()
['Apenas', 'uma', 'string', 'comum']
>>> Texto.split('string')
['Apenas uma ', ' comum']
>>>
```

Figura 19 – Método *split* em Python.

Fonte: Os autores.

Se o método s*plit* é utilizado para separar, as *strings* do Python também possuem um método que as junta, esse método é o *join*, como mostra a Figura 20.

```
>>> Texto = 'Apenas uma string comum'
>>> Texto = Texto.split()
>>> print(Texto)
['Apenas', 'uma', 'string', 'comum']
>>> ' '.join(Texto)
'Apenas uma string comum'
```

Figura 20 – Método *join* em Python.

Fonte: Os autores.

Também pode-se manipular a caixa das letras, com os métodos *lower*, *upper* e *capitalize*, as variações são mostradas na Figura 21.

```
>>> Texto = 'Apenas uma string comum'
>>> Texto.lower()
'apenas uma string comum'
>>> Texto.upper()
'APENAS UMA STRING COMUM'
>>> Texto.lower().capitalize()
'Apenas uma string comum'
```

Figura 21 – Métodos *lower*, *upper* e *capitalize* em Python.

Fonte: Os autores.

Além dos tipos e variáveis comum (*string*, *int*, *float* e *boolean*) o Python possui alguns tipos de variáveis que são denominadas como de alto nível, entre eles estão, listas, tuplas e dicionários.

Uma lista é um conjunto ordenado de valores (semelhante a um vetor em outras linguagens) que são acessados de forma sequencial ou por um índice numérico inteiro. Uma lista pode ser composta de qualquer elemento, números, caracteres, listas ou todos misturados. A Figura 22 mostra mostra os elementos do tipo lista.

```
>>> lista_vazia = []
>>> lista_numeros = [1,2,3,4,5]
>>> lista_string = ['Franca','Campinas','Rio de Janeiro']
>>> lista_mista = [34,8,'chuva',5.2,'sol',['Janeiro',22]]
>>> print(lista_vazia)
[]
>>> print(lista_numeros)
[1, 2, 3, 4, 5]
>>> print(lista_string)
['Franca', 'Campinas', 'Rio de Janeiro']
>>> print(lista_mista)
[34, 8, 'chuva', 5.2, 'sol', ['Janeiro', 22]]
```

Figura 22 – Elementos tipo lista em Python.

Fonte: Os autores.

Os elementos de uma lista podem ser facilmente acessados e manipulados, permitindo adicionar, estender ou remover elementos de uma lista, e como um elemento do tipo lista é um objeto do tipo lista, podemos ver na Figura 23 alguns exemplos de métodos utilizados para manipular elementos do tipo lista.

```
>>> lista_mista = [34,8,'chuva',5.2,'sol',['Janeiro',22]]
>>> lista_mista[0]
34
>>> lista_mista[3]
5.2
>>> lista_mista[5]
['Janeiro', 22]
>>> lista_mista.append([1,2,3])
>>> lista_mista
[34, 8, 'chuva', 5.2, 'sol', ['Janeiro', 22], [1, 2, 3]]
>>> lista_mista.extend([5,6])
>>> lista_mista
[34, 8, 'chuva', 5.2, 'sol', ['Janeiro', 22], [1, 2, 3], 5, 6]
>>> lista_mista.remove('chuva')
>>> lista_mista
[34, 8, 5.2, 'sol', ['Janeiro', 22], [1, 2, 3], 5, 6]
```

Figura 23– Manipulando lista em Python.

Fonte: Os autores.

Como pode-se observar na Figura 23, podemos obter o elemento de uma lista apenas passando como parâmetro um índice da estrutura (linhas 2 ,4 e 6). Um elemento pode ser inserido em uma lista de várias maneiras, as duas mais utilizadas são os métodos *append* e *extend*. O *append* insere o elemento que foi passado em uma nova posição da lista, independentemente do elemento, já o *extend* como seu próprio nome diz, estende a lista ou seja se for passado ao método *extend* um conjunto de elementos, cada elemento será inserido em uma posição da lista. O método *remove* exclui um elemento da lista.

A tupla é um tipo muito parecido com a lista, seus valores podem ser acessados por um índice, porém não podem ser alterados este tipo é aplicado em casos que seja mais eficiente um conjunto fixo de valores, a Figura 24 mostra um exemplo disso.

```
>>> tupla_vazia = ()
>>> tupla_mista = (1,'quatro',4.5,'mouse')
>>> tupla_mista
(1, 'quatro', 4.5, 'mouse')
>>> tupla_mista[2]
4.5
```

Figura 24 – Tuplas em Python.

Fonte: Os autores.

Dicionários são estruturas chave/valor que permite buscar um atributo através de sua chave, a diferença de dicionários com as listas é que o dicionário permite uma chave de

qualquer tipo e não índices inteiros sequenciais como as listas, como podemos observar na Figura 25.

```
>>> dic_valores = {'nome':'Lucas','idade':23,'telefones':[37028773,992837736]}
>>> dic_valores
{'idade': 23, 'telefones': [37028773, 992837736], 'nome': 'Lucas'}
>>> dic_valores['nome']
'Lucas'
```

Figura 25 – Dicionários em Python.

Fonte: Os autores.

Seus valores podem ser alterados de maneira direta, como mostra a Figura 26.

```
>>> dic_valores['idade'] = 19
>>> dic_valores
{'idade': 19, 'telefones': [37028773, 992837736], 'nome': 'Lucas'}
```

Figura 26 – Manipulando elemento do dicionário.

Fonte: Os autores.

Alguns métodos da classe dicionário, tornam ainda mais simples a manipulação desse tipo de dado, como por exemplo, o método *keys*, que retorna uma lista com todas as chaves do dicionário, o método *values*, que retorna uma lista com todos os valores e *items* que retorna uma lista de tuplas com os atributos do dicionário na estrutura chave/valor, como pode-se observar na Figura 27.

```
>>> dic_valores.keys()
['idade', 'telefones', 'nome']
>>> dic_valores.values()
[19, [37028773, 992837736], 'Lucas']
>>> dic_valores.items()
[('idade', 19), ('telefones', [37028773, 992837736]), ('nome', 'Lucas')]
>>>
```

Figura 27 – Métodos *keys*, *values* e *items* do tipo dicionário.

Fonte: Os autores.

3.3 NATURAL LANGUAGE TOOLKIT (NLTK)

Criado em 2001 por Edward Loper, Ewan Klein e Steven Pássaro, o *Natural Language Toolkit* ou NLTK é um conjunto de ferramentas para a linguagem Python, voltado para a construção de aplicações que trabalham com processamento de linguagem natural e linguística computacional.

Para poder utilizar a biblioteca NLTK deve-se importar o pacote no início de cada arquivo Python, o *import* pode ser total, que acontece quando se importa todo o pacote ou parcial quando se importa apenas uma parte da biblioteca. Perceba na Figura 28 as duas maneiras de executar uma importação.

```
>>> import nltk
>>> from nltk import word_tokenize
```

Figura 28 – Tipos de *import* da biblioteca NLTK.

Fonte: Os autores.

O processo de "tokenização" do texto, que nada mais é do que decompor o documento nas sentenças que o constituem, é uma etapa crucial em qualquer projeto que trabalhe com processamento de linguagem natural, e a biblioteca NLTK disponibiliza um método que realiza tal procedimento, o *word_tokenise*. A Figura 29, mostra o método *word_tokenize* dividindo a frase em tokens.

```
>>> texto = 'Um dia frio, um bom lugar para ler um livro'
>>> tokens_texto = word_tokenize(texto)
>>> tokens_texto
['Um', 'dia', 'frio', ',', 'um', 'bom', 'lugar', 'para', 'ler', 'um', 'livro']
```

Figura 29 – Método *word_tokenize* da biblioteca NLTK.

Fonte: Os autores.

A biblioteca também fornece um método que retorna uma lista de *stopwords*, para que possa ser feita a limpeza do texto, retirando, assim, palavras que não contém semântica, como artigos, pronomes, etc.

```
>>> nltk.corpus.stopwords.words('portuguese')
['de', 'a', 'o', 'que', 'e', 'do', 'da', 'em', 'um', 'para', 'com', 'n\
xc3\xa3o', 'uma', 'os', 'no', 'se', 'na', 'por', 'mais', 'as', 'dos', '
como', 'mas', 'ao', 'ele', 'das', '\xc3\xa0', 'seu', 'sua', 'ou', 'quan
do', 'muito', 'nos', 'j\xc3\xa1', 'eu', 'tamb\xc3\xa9m', 's\xc3\xb3', '
pelo', 'pela', 'at\xc3\xa9', 'isso', 'ela', 'entre', 'depois', 'sem', '
mesmo', 'aos', 'seus', 'quem', 'nas', 'me', 'esse', 'eles', 'voc\xc3\xa
a', 'essa', 'num', 'nem', 'suas', 'meu', '\xc3\xa0s', 'minha', 'numa',
'pelos', 'elas', 'qual', 'n\xc3\xb3s', 'lhe', 'deles', 'essas', 'esses'
, 'pelas', 'este', 'dele', 'tu', 'te', 'voc\xc3\xaas', 'vos', 'lhes', '
meus', 'minhas', 'teu', 'tua', 'teus', 'tuas', 'nosso', 'nossa', 'nosso
s', 'nossas', 'dela', 'delas', 'esta', 'estes', 'estas', 'aquele', 'aqu
ela', 'aqueles', 'aquelas', 'isto', 'aquilo', 'estou', 'est\xc3\xa1', '
estamos', 'est\xc3\xa3o', 'estive', 'esteve', 'estivemos', 'estiveram',
'estava', 'est\xc3\xa1vamos', 'estavam', 'estivera', 'estiv\xc3\xa9ramo
s', 'esteja', 'estejamos', 'estejam', 'estivesse', 'estiv\xc3\xa9ssemos
', 'estivessem', 'estiver', 'estivermos', 'estiverem', 'hei', 'h\xc3\xa
1', 'havemos', 'h\xc3\xa3o', 'houve', 'houvemos', 'houveram', 'houvera'
, 'houv\xc3\xa9ramos', 'haja', 'hajamos', 'hajam', 'houvesse', 'houv\xc
3\xa9ssemos', 'houvessem', 'houver', 'houvermos', 'houverem', 'houverei
', 'houver\xc3\xa1', 'houveremos', 'houver\xc3\xa3o', 'houveria', 'houv
er\xc3\xadamos', 'houveriam', 'sou', 'somos', 's\xc3\xa3o', 'era', '\xc
3\xa9ramos', 'eram', 'fui', 'foi', 'fomos', 'foram', 'fora', 'f\xc3\xb4
ramos', 'seja', 'sejamos', 'sejam', 'fosse', 'f\xc3\xb4ssemos', 'fossem
', 'for', 'formos', 'forem', 'serei', 'ser\xc3\xa1', 'seremos', 'ser\xc
3\xa3o', 'seria', 'ser\xc3\xadamos', 'seriam', 'tenho', 'tem', 'temos',
't\xc3\xa9m', 'tinha', 't\xc3\xadnhamos', 'tinham', 'tive', 'teve', 'ti
vemos', 'tiveram', 'tivera', 'tiv\xc3\xa9ramos', 'tenha', 'tenhamos', '
tenham', 'tivesse', 'tiv\xc3\xa9ssemos', 'tivessem', 'tiver', 'tivermos
', 'tiverem', 'terei', 'ter\xc3\xa1', 'teremos', 'ter\xc3\xa3o', 'teria
', 'ter\xc3\xadamos', 'teriam']
```

Figura 30 – Método *word_tokenize* da biblioteca NLTK.

Fonte: Os autores.

Os exemplos mostrados na Figura 30, são apenas uma pequena fração do poder da biblioteca NLTK. Mais à frente, junto ao desenvolvimento do protótipo de classificador de tweets, poderá ser observada mais profundamente a capacidade dos conjuntos de ferramentas da NLTK, no auxílio à implementação de aplicações de processamento de linguagem natural.

3.4 PROTÓTIPO DE CLASSIFICADOR DE TWEETS

Neste trabalho foi proposta a implementação de um protótipo de classificador de *tweets*, com o intuito de realizar análise de sentimento em uma rede social.

Denomina-se *Tweet* a mensagem enviada através da plataforma do microblog social Twitter. A principal característica de um *tweet* é seu tamanho máximo de 140 caracteres, portanto qualquer mensagem enviada através do microblog não pode conter mais de 140 caracteres. Portanto, qualquer mensagem enviada através do microblog não pode conter mais de 140 caractere, incluindo letras , números, caracteres especiais e espaços em branco. A Figura 31 traz o exemplo de um *tweet*, da rede social Twitter.

Figura 31– Exemplo de um *tweet*.

Fonte: Os autores.

O objetivo do protótipo é apanhar uma quantidade tweets e ser capaz de classificá-los, de forma automática, em *tweets* com sentimento positivo ou com sentimento negativo.

3.4.1 Coletando *tweets*

A coleta de *tweets*, tanto para a montagem da base de treinamento, quanto para a classificação em si, foi feita através de uma biblioteca focada em coleta de dados no Twitter, chamada *Twittersearch*. A seguir, o código utilizado para tal função.

```
def pegaTweets(self, key):
  lista_tweets = []
  tso = TwitterSearchOrder()
  tso.setKeywords([key])
  tso.setLanguage('pt')
  tso.setCount(1)

  ts = TwitterSearch(
    consumer_key = 'iaSmYN1LBd9mhMQgxBwFSw',
    consumer_secret = 'cnZsGPlyQqpl1OPb9vVHFI0g8QI5CZ96XrsNScgHw',
    access_token = '60440481-8XyGdX7GSFYjgzYlp7twaehGFlpN0BV7HGIKXvAad',
    access_token_secret = 'hTRTO2y7VTUn3x9eaCqyXABtD4VWVwQR8sHXbrX4QcFD4'
  )

  for tweet in ts.searchTweetsIterable(tso):
    processar = processamentoTexto()
    tw = processar.remover_acentos(tweet['text'].encode('utf-8').strip().strip('/n'))
    tw = processar.processatweet(tw)
    print( '@%s Twittou: %s' % ( tweet['user']['screen_name'], tweet['text'] ) )
    lista_tweets.append(tw)
    return lista_tweets
```

Figura 32 – Método pegaTweets da classe Spyder.

Fonte: Os autores.

A Figura 32 representa o único método da classe Spyder. Esse método é responsável por configurar as opções de busca dos *tweets* disponíveis na API do Twitter, que são: a palavra chave que servirá de indicador para a captura de determinado texto; a língua que este texto deve estar; e a quantidade de resultado que o algoritmo retornara por página. Neste método também é feita por mediação da biblioteca *Twittersearch* a autenticação da aplicação junto ao Twitter. Esta autenticação é feita por *tokens* e chaves secretas que são fornecidas ao desenvolvedor no momento do registro da aplicação. E, por último, o método obtém os *tweets*, aplica técnicas de processamento e retorna uma lista de *tweets* pré-processados.

3.4.2 Processamento dos *tweets*

Os *tweets* obtidos pelo método pegaTweets da classe Spyder chegam da maneira que foram escritos pelos usuários. Para um melhor resultado da classificação, os *tweets* passam por um tratamento, para a redução de ruídos, que possam vir a atrapalhar o processo de classificação.

```
def processatweet(self,tweet):
    tweet = tweet.lower()
    tweet = re.sub(r'((www\.[\s]+)|(https?://[^\s]+))','',tweet)
    tweet = re.sub(r'@[^\s]+','',tweet)
    tweet = re.sub(r'#([^\s]+)', r'\1', tweet)
    tweet = tweet.replace('rt','')
    return tweet
```

Figura 33 – Método processatweet da classe processamentoTexto.

Fonte: Os autores.

No método processatweet ilustrado na Figura 33, é realizada uma limpeza específica no texto, com o auxílio de expressão regular, retirando elementos peculiares da linguagem *do tweet*, tais como links ou *urls*, *rashtags*, menções de nomes de outros usuários como, por exemplo, @RomuloMilani e a expressão R,T que significa re-tweet.

```
def removestopwords(self,tweet):
    stopwords = nltk.corpus.stopwords.words('portuguese')
    novo_tweet = []
    for t in tweet:
        if t not in stopwords and t not in string.punctuation:
            novo_tweet.append(t)
    return novo_tweet
```

Figura 34 – Método removestopwords da classe processamentoTexto.

Fonte: Os autores.

As chamadas *stopwords* e caracteres de pontuação, são removidos com o método removestopwords, que alimenta sua lista de palavras que não contém semântica do corpus de *stopwords* em português, que a biblioteca NLTK fornece, e com o método *ponctuation* do objeto *string*, que retorna uma lista de caracteres de pontuação. Esse processo pode ser observado na Figura 34.

```
def tokenizacao(self,tweet):
    texto_tokenizado = nltk.word_tokenize(tweet)
    return texto_tokenizado
```

Figura 35 – Método tokenizacao da classe processamentoTexto.

Fonte: Os autores.

O Método de tokenização, como mostra a Figura 35, recebe um tweet e realiza o processo de transformar as frases em tokens.

O último método da classe processamentoTexto, que podemos observar na Figura 36, tem como função, remover os acentos das sentenças, com a aplicação do método *normalize* do pacote unicodedata. São aplicadas técnicas de decomposição de compatibilidade para a substituição dos caracteres.

```
def remover_acentos(self, txt): #função que retira os acentos
    return normalize('NFKD', txt.decode('utf-8')).encode('ASCII','ignore')
```

Figura 36 – Método remover_acentos da classe processamentoTexto.

Fonte: Os autores

3.4.3 Classificação Supervisionada

A classificação é a tarefa de escolher o correto rótulo de classe para uma determinada entrada. Em tarefas básicas de classificação, cada entrada é considerada isoladamente de todas as outras entradas, e o conjunto de etiquetas é definido com antecedência (BIRD; KLEIN;LOPER,2009).

O modelo de classificação abordado neste trabalho é denominado supervisionado. A classificação supervisionada acontece quando é construída com apoio em uma base de dados de treinamento que contém o rótulo correto para cada entrada. A Figura 37 ilustra todo o processo de classificação supervisionada.

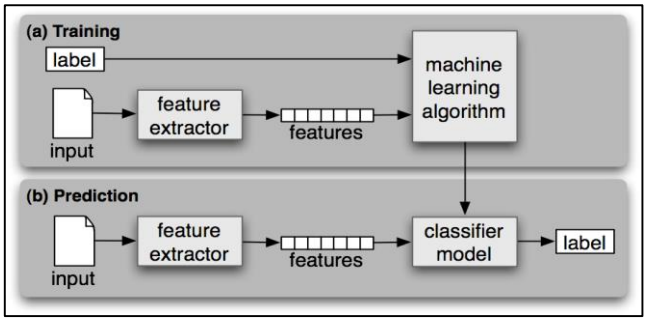

Figura 37 – Modelo utilizado pela classificação supervisionada.

Fonte: LOPES, 2009, p.67.

3.4.4 Base de Treinamento

Para realizar a classificação supervisionada de forma autônoma, é necessário um conjunto de *tweets* classificados manualmente. Na implementação para o treinamento do protótipo foram utilizados 500 *tweets* positivos e 500 *tweets* negativos, números considerados razoáveis. Para se obter maior precisão deve-se usar um conjunto maior.

```
tweets_positivos = [('Eu gosto deste carro', 'positivo'),
                    ('legal ver voce aqui', 'positivo'),
                    ('noite tava boa pra dizer melhor exelente','positivo'),
                    ('sim vou enviar agora','positivo'),
                    ('sentindo bem avontade ','positivo'),
                    ('esporte e espetacular', 'positivo')]
```

Figura 38 – Lista de tuplas contendo tweets positivos.

Fonte: Os autores.

```
tweets_negativos = [('voce foi mau na prova', 'negativo'),
                    ('porcaria ficou toda grudada aparelho','negativo'),
                    ('serio não gostei','negativo'),
                    ('horrimel ficar sozinho', 'negativo'),
                    ('o que ele fez foi inadmissivel', 'negativo'),
                    ('este computador e inutil', 'negativo')]
```

Figura 39 – Lista de tuplas contendo tweets negativos.

Fonte: Os autores.

As Figuras 38 e 39 exemplificam a estrutura utilizada no protótipo para manipular os *tweets* classificados manualmente. Foram geradas duas listas de tuplas com a estrutura tweet/etiqueta. Observa-se que, a partir das duas listas, podemos gerar uma terceira lista, contendo todas as etiquetas.

3.4.5 Implementação do classificador

O primeiro passo para o processo de treinamento do algoritmo é a extração das características do documento. E para extrair as características, necessita-se de todas as palavras contidas nele. A função do método pega_palavras_tweet, que temos na Figura 40, é

exatamente esta. Dada a lista de *tweets* de treinamento, o método retira e cria uma lista com todas as palavras contidas no documento.

```
def pega_palavras_tweet(self,list_tweet):
    todas_palavras = []
    for (palavra, sentimento) in list_tweet:
        todas_palavras.extend(palavra)
    return todas_palavras
```

Figura 40 – Método que retira todas as palavras da lista de treinamento.

Fonte: Os autores.

Após obter todas as palavras contidas no documento, é necessário obter as palavras mais frequentes desta lista de palavra. O método pega_frequencia_de_palavras, da Figura 41, retorna essas palavras.

```
def pega_frequencia_das_palavras(self,lista_palavras):
    list_frequencia = nltk.FreqDist(lista_palavras)
    palavras_frequentes = list_frequencia.keys()[:2000]
    return palavras_frequentes
```

Figura 41 – Método retorna as 2000 palavras mais frequentes.

Fonte: Os autores.

A função *FreqDist* da biblioteca NLTK, recebe a lista de palavras e retorna um dicionário palavra/frequência, como pode ser visto na Figura 42, e com o método *Keys* se extrai apenas o índice do dicionário, formando assim uma lista das 2000 palavras mais frequentes.

```
<FreqDist:
  'noite': 35,
  'amanha': 33,
  'aqui': 32,
  'queria': 26,
  'quero': 26,
  'feliz': 21,
  'gosto': 20,
  'agora': 19,
  'deus': 19,
  'fazer': 19,
  ...>
```

Figura 42 – Dicionário resultado do método *FreqDist* do NLTK.

Fonte: Os autores

Após obter o conjunto de treinamento e as palavras mais frequentes desse conjunto, pode-se extrair as características do documento. A função extrair_caracteristicas recebe o conjunto de treinamento e compara palavra por palavra com as palavras mais frequentes, como pode-se observar na Figura 43.

```
def extrair_caracteristicas(self,documento):
    document_words = documento
    caracteristicas = {}
    for palavra in palavras_frequentes:
        caracteristicas['Contem(%s)' %palavra] = (palavra in document_words)
    return caracteristicas
```

Figura 43 – Método de extração de características.

Fonte: Os autores.

O resultado do método ilustrado na Figura 44 é um dicionário, que informa se a palavra do documento contém as palavras mais frequentes. Por exemplo se um dos *tweets* apresentar as palavras [‘feliz’,’ hoje’, ‘bicicleta’], a saída resultante seria assim:

```
{'Contem(luta)': False,
 'Contem(bicicleta)': True,
 'Contem(sonho)': False,
 'Contem(hackeia)': False,
 'Contem(acordar)': False,
 'Contem(confoavel)': False,
 'Contem(hoje)': True,
 'Contem(feliz)': True,
 'Contem(site)': False
 ...}
```

Figura 44 – Dicionário resultado do método extrair_caracteristicas.

Fonte: Os autores.

Após obter o extrator de características, ele será utilizado para criar o conjunto de treinamento para o classificador *Naive Bayes*. Obtém-se esse conjunto com o método *apply_features* da biblioteca NLTK, método que recebe a função que extrai as características e a lista de *tweets* de treinamento, e retorna uma lista de tuplas, contendo cada tupla uma estrutura dicionário de característica/etiqueta, como é ilustrado na Figura 45.

```
[({'Contem(luta)': False,
   'Contem(bicicleta)': True,
   'Contem(sonho)': True,
   'Contem(hackeia)': False,
   ...
   'Contem(feliz)': True}, 'positivo'),
 ({'Contem(mau)': False,
   'Contem(jogo)': False,
   'Contem(futebol)': True,
   'Contem(gol)': True,
   ...
   'Contem(trave)': False}, 'negativo'),
  ...]
```

Figura 45 – Conjunto de treinamento, retorno do método *apply_features.*

Fonte: Os autores.

Com o conjunto de treinamento pode-se treinar o classificador *Naive Bayes*, através do método *train*. A Figura 46 mostra como inicia-se o treinamento.

```
classificador = nltk.NaiveBayesClassifier.train(conjunto_treino)
```

Figura 46 – Treinando o classificador *Naive Bayes*.

Fonte: Os autores.

Com o classificador treinado pode-se obter informações interessantes sobre o mesmo. Uma delas é apresentar as características mais relevantes, com o método *show_most_informative_features*.

```
print(classificador.show_most_informative_features())

  Contem(queria) = True           negati : positi =     17.0 : 1.0
Contem(parabens) = True           positi : negati =     11.7 : 1.0
  Contem(triste) = True           negati : positi =     11.0 : 1.0
     Contem(boa) = True           positi : negati =     10.7 : 1.0
   Contem(noite) = True           positi : negati =      9.3 : 1.0
Contem(saudades) = True           negati : positi =      7.7 : 1.0
   Contem(legal) = True           positi : negati =      7.0 : 1.0
     Contem(nao) = True           negati : positi =      7.0 : 1.0
    Contem(cara) = True           negati : positi =      6.3 : 1.0
 Contem(saudade) = True           negati : positi =      5.4 : 1.0
   Contem(final) = True           negati : positi =      5.0 : 1.0
    Contem(foda) = True           negati : positi =      4.3 : 1.0
   Contem(odeio) = True           negati : positi =      4.3 : 1.0
   Contem(ainda) = True           negati : positi =      3.8 : 1.0
  Contem(melhor) = True           positi : negati =      3.8 : 1.0
     Contem(mal) = True           negati : positi =      3.8 : 1.0
   Contem(quase) = True           positi : negati =      3.7 : 1.0
```

Figura 47 – Características relevantes do classificador.

Fonte: Os autores.

Como se pode observar na figura 47, de acordo com a base de treinamento, *tweets* que contenham a palavra “parabéns” são positivos 11.7 vezes mais do que eles são negativos, enquanto, por exemplo, *tweets* que contenham a palavra “saudade” são negativos 5.4 vezes mais do que do que elas são positivas. Este cálculo é conhecido como razão de verossimilhança.

O pacote NLTK também traz um método onde é possível medir o grau de precisão do classificador em relação ao conjunto teste. A função *classify.accuracy*, com base no classificador e em um conjunto teste, calcula a precisão do algoritmo, onde 0.00 é o mínimo e 1.00 é o máximo. A Figura 48 mostra a precisão do algoritmo aplicado.

```
>>> nltk.classify.accuracy(classificador, conjunto_teste)
0.45
```

Figura 48 – Método para o cálculo de precisão do classificador.

Fonte: Os autores.

Após passar por cada etapa descrita, o algoritmo está pronto para realizar a classificação de um documento, no caso, um *tweet*. A Figura 49, mostra um *tweet* sendo classificado.

```
>>> Tweet = 'É mais do que clara a intenção malévola da Veja de interferir
de forma desonesta e desleal nos resultados das eleições.'
>>> treino = Treino()
>>> classificador.classify(treino.extrair_caracteristicas(Tweet))
'negativo'
```

Figura 49 – Classificação de um *tweet*.

Fonte: Os autores.

O método *classify* recebe as características extraídas do *tweet* e realiza o processo de classificação a partir dos dados do algoritmo *Naive Bayes* treinado. Como pode ser observado na figura 48, o *tweet* foi classificado como negativo, pois as palavras ‘malévola’, ‘desonesta’ e ‘desleal’, apresentam uma grande probabilidade de serem associadas a *tweets* negativos.

3.4.6 Resultados Obtidos

Tabela 1 – Análise manual versus análise do algoritmo.

SENTIMENTO	NUMERO TOTAL DA BASE DE TESTE
POSITIVO	50 tweets
NEGATIVO	50 tweets
RESULTADO DO PROCESSO DE CLASSIFICAÇÃO	
POSITIVO	44 tweets
NEGATIVO	56 tweets
ACERTOS	74
ERROS	26

Fonte: Os autores.

A tabela 1 retrata o resultado de uma análise feita com o algoritmo de classificação em um cenário onde foram classificados, previamente, de forma manual, cem *tweets*, onde cinquenta eram positivos e cinquenta negativos. O resultado foi que de cinquenta *tweets* positivos o algoritmo acertou 34 e errou 16, e dos outros cinquenta tweets que eram negativos o algoritmo acertou 40 e errou 10, classificando assim, 74 *tweets* de maneira correta e errando apenas 26 *tweets*.

Outro teste realizado ocorreu em um panorama com base no segundo turno da eleição presidencial de 2014. Foi feita a análise de sentimento de 900 *tweets* em relação a Presidente Dilma Rousseff, logo após o anúncio de sua reeleição. O resultado obtido pelo algoritmo foi de 486 *tweets* positivos contra 414 *tweets* negativos. O resultado está apresentado, de maneira gráfica, na figura 50.

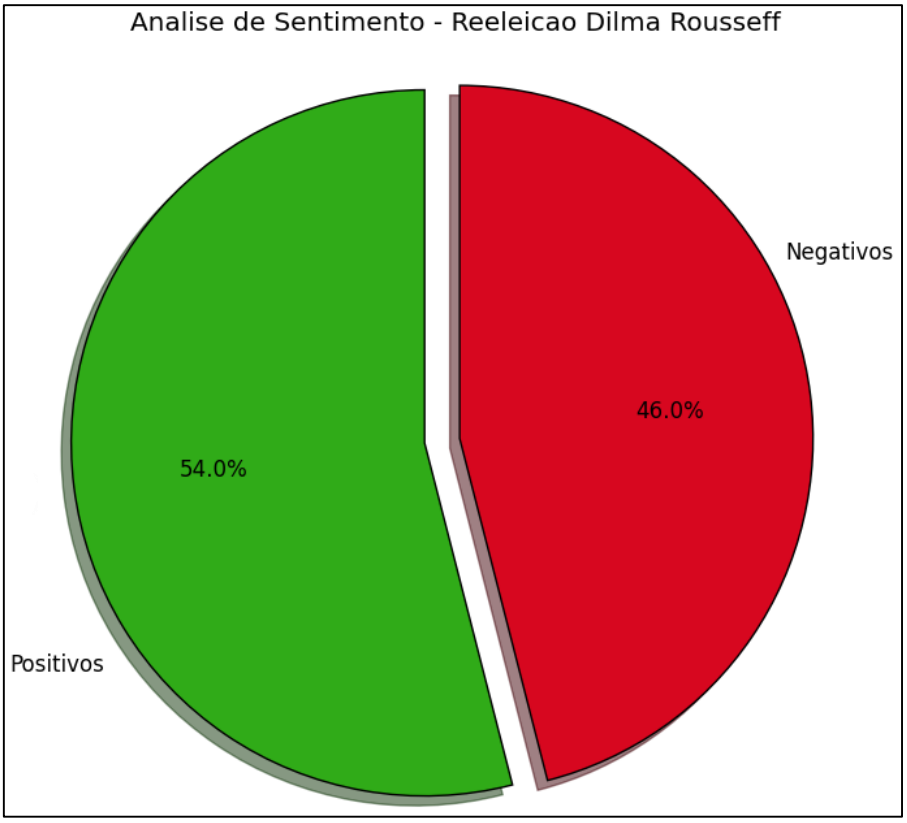

Figura 50 – Análise de sentimentos referentes a reeleição de Dilma Rousseff.
Fonte: Os autores.

O resultado do teste feito no Twitter pelo algoritmo se comparado às pesquisas que se estenderam durante todo o segundo turno feitas pelos principais órgãos de pesquisa do País, mostram a mesma paridade nas intenções de voto. As figuras 51 e 52, representam as pesquisas feitas pelo Ibope e Datafolha.

Figura 51 – Pesquisa de intenção de votos.

Fonte: IBOPE, 2014.

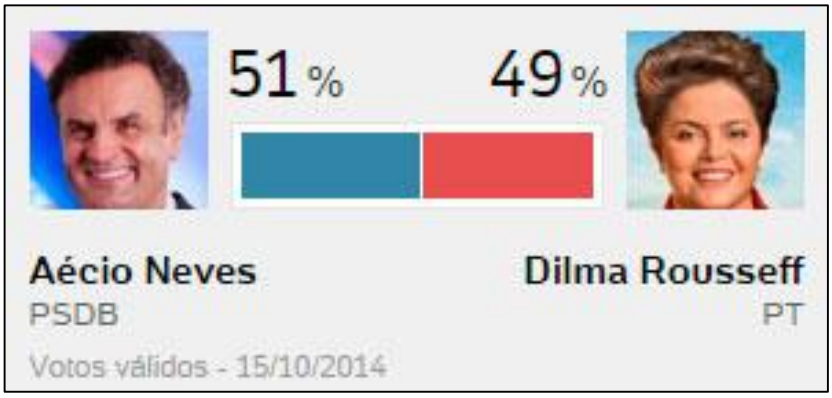

Figura 52 – Pesquisa de intenção de votos.

Fonte: DATAFOLHA, 2014.

CONCLUSÃO

O presente trabalho atingiu os objetivos propostos no início do desenvolvimento, que eram: o estudo da tecnologia de mineração de texto; e a implementação de um protótipo de classificador que fizesse de forma autônoma a classificação de pequenos textos retirados de uma rede social.

O foco se manteve em apresentar de maneira completa os principais procedimentos e tecnologias que abrangem o processo de mineração de texto.

A aplicação prática, atingiu os resultados esperados, coletando, processando, classificando e apresentando resultados a partir de um conjunto de dados não estruturados. Durante a construção do protótipo de classificador pode-se observar o quão trabalhoso é lidar com dados que se encontram em linguagem natural.

A base de material escolhido para o trabalho, também se mostrou como um grande desafio. Além do fato de se encontrar em linguagem natural, também se apresentava em uma linguagem informal, contento muitos erros ortográficos, gírias, abreviações e vários elementos oriundos da própria rede social como, por exemplo, as *rashtags*.

Pode-se concluir que a linguagem Python superou as expectativas, mostrando ser uma linguagem flexível e produtiva, simplificando o processo de desenvolvimento com sua sintaxe simples e seu alto poder de manipulação de *strings* e listas. Somada aos recursos da biblioteca NLTK, torna-se uma ferramenta de altíssimo nível para a construção de aplicações que têm, como objetivo, o processamento de linguagem natural.

Apesar de ser utilizada uma pequena fração de sua capacidade, o kit de desenvolvimento NLTK, contribuiu de forma crucial na aplicação, fornecendo ferramentas para o auxílio do processamento e classificação do material. O kit NLTK tem a maioria das suas funções voltadas para a língua inglesa, dificultando, assim, a usabilidade de suas ferramentas em dados textuais que estejam em outras línguas. Sua documentação é abundante tornando seu aprendizado fácil e didático.

Por se tratar de um protótipo, ainda há muito o que ser estudado e aprimorado. A seguir, é apresentada uma lista com melhoramentos e futuros projetos:

- criação de um dicionário, com o intuito de diminuir a perda da semântica de palavras abreviadas e/ou gírias;
- implementação de um banco de dados, para melhor organização, tratamento e recuperação dos dados coletados;
- estudo e aplicação de outros algoritmos de mineração de texto, que trabalhem com a análise gramatical do documento textual;
- o desenvolvimento de um sistema de análise de sentimento de produtos, que identifiquem padrões de pessoas que possam vim a se tornar clientes, com a finalidade de marketing personalizado;
- desenvolvimento de um sistema de classificação de textos médicos, que extraia informações de documentos, livros, diagnósticos antigos, com a finalidade de auxiliar o profissional da medicina a fazer diagnósticos mais precisos;
- contribuir com o projeto NLTK, estudando a biblioteca e projetando funções que trabalhem, também, com a língua portuguesa; auxiliando o desenvolvimento da comunidade.

REFERÊNCIAS

ARANHA, C. N. **Uma abordagem de pré-processamento para mineração de textos em português: sob o enfoque da inteligência computacional.** 2007. 144 f. Tese (Doutorado em Engenharia Elétrica) – Pontífice Universidade Católica, Rio de Janeiro.

ARDÖ, A. **Focused Crawling in the ALVIS Semantic Search Engin**e. In Proceedings of 2nd European Semantic Web Conference (ESWC). 2005. Heraklion, Greece. Disponível em: <http://www.eit.lth.se/fileadmin/eit/home/hs.aar/Publ/ESWC2005posterArdo.pdf> Acesso em: 30 de jul. 2014.

BASTOS, V. M. **Ambiente de descoberta de conhecimento na web para língua portuguesa**. 2006. 124 f. Tese (Doutorado em Ciências em Engenharia Civil) – Pontífice Universidade Católica, Rio de Janeiro.

BIRD, S.; KLEIN E.; LOPER, E. **Natural Language Processing with Python - Analyzing Text with the Natural Language Toolkit.** Disponível em: <http://www.nltk.org/book/>. Acesso em 21: jun. 2014.

BORGES, L. E. **Phyton para desenvolvedores.** Rio de Janeiro: Edição do autor, 2009.

BRAGA, L. P. V. **Introdução à mineração de dados**. 2 ed. Rio de Janeiro: E-Papers, 2005.

CARRILHO, J. R. J. **Desenvolvimento de uma metodologia para mineração de textos.** 2007. 96 f. Dissertação (Mestrado em Engenharia Elétrica) - Pontífice Universidade Católica, Rio de Janeiro.

FAYYAD,U. M. **Advanced in knowledge discovery and data mining.** California: Aaai, 1996.

FELDMAN, R.; SANGER, J. **The text mining handbook.** Reino Unido: Cambridge University Press, 2007.

GOLDSCHMIDT, R.; PASSOS, E. **Data Mining: um guia prático**. Rio de Janeiro: Campus, 2005.

KHONCHADY, M. **Text Mining Application Programming**. Estados Unidos: Charles River Media, 2006.

LOPES, M. C. S. **Mineração de dados textuais utilizando técnicas de clustering para o idioma português**. 2004. 180 f. Tese (Doutorado em Ciências em Engenharia Civil) - Pontífice Universidade Católica, Rio de Janeiro.

LUCAS, M. **Mining in textual mountains**. Mapa Mundi Magazine, Disponível em <http://mappa.mundi.net/trip-m/hearst/>. Acesso em 23 jun. 2014.

LUGER, G.F. **Artificial Intelligence: Structures and Strategies for Complex Problem Solving.** 5.ed. Estados Unidos: Pearson Adisson Wesley, 2004.

SANTOS, L. M. **Protótipo para mineração de opinião em redes sociais: estudo de casos selecionados usando o twitter.** 2009. Trabalho de Conclusão de Curso (Graduação em Ciência da Computação) – Universidade de Lavras, Lavras.

SOARES, F. A**. Mineração de textos na coleta inteligente de dados na web.** 2008. 120 f. Dissertação (Mestrado em Engenharia Elétrica) – Pontífice Universidade Católica, Rio de Janeiro.

SPIEGEL, M.R. **Estatística e Probabilidade.** Rio de Janeiro: Makron Books, 2003.